Chapter 6 ▪ Interfaces, Lambda Expressions, and Inner Classes

You have now learned about classes and inheritance, the key concepts of object-oriented programming in Java. This chapter shows you several advanced techniques that are commonly used. Despite their less obvious nature, you will need to master them to complete your Java tool chest.

The first technique, called *interfaces*, is a way of describing *what* classes should do, without specifying *how* they should do it. A class can *implement* one or more interfaces. You can then use objects of these implementing classes whenever conformance to the interface is required. After discussing interfaces, we move on to *lambda expressions*, a concise way to create blocks of code that can be executed at a later point in time. Using lambda expressions, you can express code that uses callbacks or variable behavior in an elegant and concise fashion.

We then discuss the mechanism of *inner classes*. Inner classes are technically somewhat complex—they are defined inside other classes, and their methods can access the fields of the surrounding class. Inner classes are useful when you design collections of cooperating classes.

This chapter concludes with a discussion of *proxies*, objects that implement arbitrary interfaces. A proxy is a very specialized construct that is useful for building system-level tools. You can safely skip that section on first reading.

6.1. Interfaces

In the following sections, you will learn what Java interfaces are and how to use them. You will also find out how interfaces have been made more powerful in recent versions of Java.

6.1.1. The Interface Concept

In the Java programming language, an interface is not a class but a set of *requirements* for the classes that want to conform to the interface.

Typically, the supplier of some service states: “If your class conforms to a particular interface, then I’ll perform the service.” Let’s look at a concrete example. The `sort` method of the `Arrays` class promises to sort an array of objects, but under one condition: The objects must belong to classes that *implement* the `Comparable` interface.

Here is what the `Comparable` interface looks like:

```
public interface Comparable {
    int compareTo(Object other);
}
```

In the interface, the `compareTo` method is *abstract*—it has no implementation. A class that implements the `Comparable` interface needs to have a `compareTo` method, and the method must have an `Object` parameter and return an integer. Otherwise, the class is also abstract—that is, you cannot construct any objects.



Note: As of Java 5, the `Comparable` interface has been enhanced to be a generic type.

```
public interface Comparable<T> {
    int compareTo(T other); // parameter has type T
}
```

For example, a class that implements `Comparable<Employee>` must supply a method

```
int compareTo(Employee other)
```

You can still use the "raw" `Comparable` type without a type parameter. Then the `compareTo` method has a parameter of type `Object`, and you have to manually cast that parameter of the `compareTo` method to the desired type. I will do just that for a little while so that you don't have to worry about two new concepts at the same time.

All methods of an interface are automatically `public`. For that reason, it is not necessary to supply the keyword `public` when declaring a method in an interface.

Of course, there is an additional requirement that the interface syntax cannot express: When calling `x.compareTo(y)`, the `compareTo` method must *compare* the two objects and return an indication whether `x` or `y` is larger. The method is supposed to return a negative number if `x` is smaller than `y`, zero if they are equal, and a positive number otherwise.

This particular interface has a single method. Some interfaces have multiple methods. As you will see later, interfaces can also define constants. What is more important, however, is what interfaces *cannot* supply. Interfaces never have instance fields. Before Java 8, all methods in an interface were abstract. As you will see in **Section 6.1.4** and **Section 6.1.5**, it is now possible to have other methods in interfaces. Of course, those methods cannot refer to instance fields—interfaces don't have any.

Now, suppose we want to use the `sort` method of the `Arrays` class to sort an array of `Employee` objects. Then the `Employee` class must *implement* the

`Comparable` interface.

To make a class implement an interface, you carry out two steps:

1. You declare that your class intends to implement the given interface.
2. You supply definitions for all methods in the interface.

To declare that a class implements an interface, use the `implements` keyword:

```
class Employee implements Comparable
```

Of course, now the `Employee` class needs to supply the `compareTo` method. Let's suppose that we want to compare employees by their salary. Here is an implementation of the `compareTo` method:

```
public int compareTo(Object otherObject) {
    Employee other = (Employee) otherObject;
    return Double.compare(salary, other.salary);
}
```

Here, we use the static `Double.compare` method that returns a negative if the first argument is less than the second argument, `0` if they are equal, and a positive value otherwise.

Caution: In the interface declaration, the `compareTo` method was not declared `public` because all methods in an *interface* are automatically public. However, when implementing the interface, you must declare the method as `public`. Otherwise, the compiler assumes that the method has package access—the default for a *class*. The compiler then complains that you're trying to supply a more restrictive access privilege.

We can do a little better by supplying a type parameter for the generic `Comparable` interface:

```
class Employee implements Comparable<Employee> {
    public int compareTo(Employee other) {
       return Double.compare(salary, other.salary);
    }
    . . .
}
```

Note that the unsightly cast of the `Object` parameter has gone away.



Tip: The `compareTo` method of the `Comparable` interface returns an integer. If the objects are not equal, it does not matter what negative or positive value you return. This flexibility can be useful when you are comparing integer fields. For example, suppose each employee has a unique integer `id`

and you want to sort by the employee ID number. Then you can simply return `id - other.id`. That value will be some negative value if the first ID number is less than the other, `0` if they are the same ID, and some positive value otherwise. However, there is one caveat: The range of the integers must be small enough so that the subtraction does not overflow. If you know that the IDs are not negative or that their absolute value is at most `(Integer.MAX_VALUE - 1) / 2`, you are safe. Otherwise, call the static `Integer.compare` method.

Of course, the subtraction trick doesn't work for floating-point numbers. The difference `salary - other.salary` can round to `0` if the salaries are close together but not identical. The call `Double.compare(x, y)` simply returns `-1` if `x < y` or `1` if `x > y`.



Note: The documentation of the `Comparable` interface suggests that the `compareTo` method should be compatible with the `equals` method. That is, `x.compareTo(y)` should be zero exactly when `x.equals(y)`. Most classes in the Java API that implement `Comparable` follow this advice.

A notable exception is `BigDecimal`. Consider `x = new BigDecimal("1.0")` and `y = new BigDecimal("1.00")`. Then `x.equals(y)` is `false` because the numbers differ in precision. But `x.compareTo(y)` is zero. Ideally, it shouldn't be, but there is no obvious way of deciding which one should come first.

Another exception is `StringBuilder`, which implements `Comparable` but does not override `equals`:

```
StringBuilder x = new StringBuilder("Hello");
StringBuilder y = new StringBuilder("Hello");
x.equals(y) // false
x.compareTo(y) // 0
```



Caution: There are minor differences between comparison operators with `double` operands and the corresponding methods of the `Double` class.

The first issue is negative zero, or `-0.0`. When compared with a relational operator, it is indistinguishable from `0.0`:

```
0.0 < -0.0 // false
-0.0 < 0.0 // false
```

But `Double.compare` can tell them apart:

```
Double.compare(-0.0, 0.0) // -1
```

The other issue is `Double.NaN`. Any comparison with a relational operator where an operand is `NaN` returns `false`:

```
Double.NaN < Double.NaN // false
```

However, `Double.compare` behaves differently:

```
Double.compare(Double.NaN, Double.NaN) // 0
```

Remarkably, `Double.NaN` is deemed larger than `Double.POSITIVE_INFINITY:`

```
Double.compare(Double.POSITIVE_INFINITY, Double.NaN) // -1
```

Mathematically speaking, the `compare` method provides a "total ordering", but the `<` operator does not.

Now you saw what a class must do to avail itself of the sorting service—it must implement a `compareTo` method. That's eminently reasonable. There needs to be some way for the `sort` method to compare objects. But why can't the `Employee` class simply provide a `compareTo` method without implementing the `Comparable` interface?

The reason for interfaces is that the Java programming language is *strongly typed*. When making a method call, the compiler needs to be able to check that the method actually exists. Somewhere in the `sort` method will be statements like this:

```
if (a[i].compareTo(a[j]) > 0) {
    // rearrange a[i] and a[j]
    . . .
}
```

The compiler must know that `a[i]` actually has a `compareTo` method. If `a` is an array of `Comparable` objects, then the existence of the method is assured because every class that implements the `Comparable` interface must supply the method.



Note: You would expect that the `sort` method in the `Arrays` class is defined to accept a `Comparable[]` array so that the compiler can complain if anyone ever calls `sort` with an array whose element type doesn't implement the `Comparable` interface. Sadly, that is not the case. Instead, the `sort` method accepts an `Object[]` array and uses a clumsy cast:

```
// approach used in the standard library--not recommended
if (((Comparable) a[i]).compareTo(a[j]) > 0) {
    // rearrange a[i] and a[j]
    . . .
}
```

If `a[i]` does not belong to a class that implements the `Comparable` interface, the virtual machine throws an exception.

Listing 6.1 presents the full code for sorting an array of instances of the class `Employee` (**Listing 6.2**).

Listing 6.1 v1ch06/interfaces/InterfacesDemo.java

```
1  package v1ch06.interfaces;
2
3  import module java.base;
4
5  /**
6   * This program demonstrates the use of the Comparable interface.
7   */
8  class InterfacesDemo {
9      void main() {
10         var staff = new Employee[3];
11
12         staff[0] = new Employee("Harry Hacker", 35000);
13         staff[1] = new Employee("Carl Cracker", 75000);
14         staff[2] = new Employee("Tony Tester", 38000);
15
16         Arrays.sort(staff);
17
18         // print out information about all Employee objects
19         for (Employee e : staff)
20             IO.println("name=" + e.getName() + ",salary=" + e.getSalary());
21     }
22 }
```

Listing 6.2 v1ch06/interfaces/Employee.java

```
1  package v1ch06.interfaces;
2
3  /**
4   * The familiar Employee class, implementing the Comparable interface.
5   */
6  public class Employee implements Comparable<Employee> {
7      private String name;
8      private double salary;
9
10     public Employee(String name, double salary) {
11         this.name = name;
12         this.salary = salary;
13     }
```

```
14
15      public String getName() {
16          return name;
17      }
18
19      public double getSalary() {
20          return salary;
21      }
22
23      public void raiseSalary(double byPercent) {
24          double raise = salary * byPercent / 100;
25          salary += raise;
26      }
27
28      /**
29       * Compares employees by salary.
30       * @param other another Employee object
31       * @return a negative value if this employee has a lower salary than other,
32       *         0 if the salaries are the same, a positive value otherwise
33       */
34      public int compareTo(Employee other) {
35          return Double.compare(salary, other.salary);
36      }
37  }
```

java.lang.Comparable<T> **1.0**

- `int compareTo(T other)`
  compares this object with `other` and returns a negative integer if this object is less than `other`, zero if they are equal, and a positive integer otherwise.

`java.util.Arrays` **1.2**

- `static void sort(Object[] a)`
  sorts the elements in the array `a`. All elements in the array must belong to classes that implement the `Comparable` interface, and they must all be comparable to each other.

`java.lang.Integer` **1.0**

- `static int compare(int x, int y)` **7**
  returns a negative integer if `x < y`, zero if `x` and `y` are equal, and a positive integer otherwise.

`java.lang.Double` **1.0**

- `static int compare(double x, double y)` **1.4**
  returns a negative integer if `x < y`, zero if `x` and `y` are equal, and a positive integer otherwise.



Note: According to the language standard: "The implementor must ensure `sgn(x.compareTo(y)) = -sgn(y.compareTo(x))` for all `x` and `y`. (This implies that `x.compareTo(y)` must throw an exception if `y.compareTo(x)` throws an exception.)" Here, `sgn` is the *sign* of a number: `sgn(n)` is –1 if `n` is negative, `0` if `n` equals `0`, and `1` if `n` is positive. In plain English, if you flip the arguments of `compareTo`, the sign (but not necessarily the actual value) of the result must also flip.

As with the `equals` method, problems can arise when inheritance comes into play.

Since `Manager` extends `Employee`, it implements `Comparable<Employee>` and not `Comparable<Manager>`. If `Manager` chooses to override `compareTo`, it must be prepared to compare managers to employees. It can't simply cast an employee to a manager:

```
class Manager extends Employee {
    public int compareTo(Employee other) {
        Manager otherManager = (Manager) other; // NO
       . . .
    }
    . . .
}
```

That violates the "antisymmetry" rule. If `x` is an `Employee` and `y` is a `Manager`, then the call `x.compareTo(y)` doesn't throw an exception—it simply compares `x` and `y` as employees. But the reverse, `y.compareTo(x)`, throws a `ClassCastException`.

This is the same situation as with the `equals` method discussed in **Chapter 5**, and the remedy is the same. There are two distinct scenarios.

If subclasses have different notions of comparison, then you should outlaw comparison of objects that belong to different classes. Each `compareTo` method should start out with the test

```
if (getClass() != other.getClass()) throw new ClassCastException();
```

If there is a common algorithm for comparing subclass objects, simply provide a single `compareTo` method in the superclass and declare it as `final`.

For example, suppose you want managers to be better than regular employees, regardless of salary. What about other subclasses such as `Executive` and `Secretary`? If you need to establish a pecking order, supply a method such as `rank` in the `Employee` class. Have each subclass override `rank`,

and implement a single `compareTo` method that takes the `rank` values into account.

6.1.2. Properties of Interfaces

Interfaces are not classes. In particular, you can never use the `new` operator to instantiate an interface:

```
x = new Comparable(. . .); // ERROR
```

However, even though you can't construct interface objects, you can still declare interface variables.

```
Comparable x; // OK
```

An interface variable must refer to an object of a class that implements the interface:

```
x = new Employee(. . .); // OK provided Employee implements Comparable
```

Next, just as you use `instanceof` to check whether an object is of a specific class, you can use `instanceof` to check whether an object implements an interface:

```
if (anObject instanceof Comparable) { . . . }
```

Just as you can build hierarchies of classes, you can extend interfaces. This allows for multiple chains of interfaces that go from a greater degree of generality to a greater degree of specialization. For example, suppose you had an interface called `Moveable`.

```
public interface Moveable {
    void move(double x, double y);
}
```

Then, you could imagine an interface called `Powered` that extends it:

```
public interface Powered extends Moveable {
    double milesPerGallon();
}
```

Although you cannot put instance fields in an interface, you can supply constants in them. For example:

```
public interface Powered extends Moveable {
    double milesPerGallon();
    double SPEED_LIMIT = 95; // a public static final constant
}
```

Just as methods in an interface are automatically `public`, fields are always `public static final`.



Note: It is legal to tag interface methods as `public`, and fields as `public static final`. Some programmers do that, either out of habit or for greater clarity. However, the Java Language Specification recommends that the redundant keywords not be supplied, and I follow that recommendation.

While each class can have only one superclass, classes can implement *multiple* interfaces. This gives you the maximum amount of flexibility in defining a class's behavior. For example, the Java programming language has an important interface built into it, called `Cloneable`. (This interface is discussed in detail in Section 6.1.9.) If your class implements `Cloneable`, the `clone` method in the `Object` class will make an exact copy of your class's objects. If you want both cloneability and comparability, simply implement both interfaces. Use commas to separate the interfaces that you want to implement:

```
class Employee implements Cloneable, Comparable
```



Note: Records and enumeration classes cannot extend other classes (since they implicitly extend the `Record` and `Enum` class). However, they can implement interfaces.



Note: Interfaces can be `sealed`. As with sealed classes, the direct subtypes (which can be classes or interfaces) must be declared in a `permits` clause or be located in the same source file.

6.1.3. Interfaces and Abstract Classes

If you read the section about abstract classes in Chapter 5, you may wonder why the designers of the Java programming language bothered with introducing the concept of interfaces. Why can't `Comparable` simply be an abstract class:

```
abstract class Comparable { // why not?
    public abstract int compareTo(Object other);
}
```

The `Employee` class would then simply extend this abstract class and supply the `compareTo` method:

```
class Employee extends Comparable { // why not?
    public int compareTo(Object other) { . . . }
}
```

There is, unfortunately, a major problem with using an abstract base class to express a generic property. A class can only extend a single class. Suppose the `Employee` class already extends a different class, say, `Person`. Then it can't extend a second class.

```
class Employee extends Person, Comparable // ERROR
```

But each class can implement as many interfaces as it likes:

```
class Employee extends Person implements Comparable // OK
```

Other programming languages, in particular C++, allow a class to have more than one superclass. This feature is called *multiple inheritance*. The designers of Java chose not to support multiple inheritance, because it makes the language either very complex (as in C++) or less efficient (as in Eiffel).

Instead, interfaces afford most of the benefits of multiple inheritance while avoiding the complexities and inefficiencies.



Note: C++ has multiple inheritance and all the complications that come with it, such as virtual base classes, dominance rules, and transverse pointer casts. Few C++ programmers use multiple inheritance, and some say it should never be used. Other programmers recommend using multiple inheritance only for the "mix-in" style of inheritance. In the mix-in style, a primary superclass describes the parent object, and additional superclasses (the so-called mix-ins) may supply auxiliary characteristics. That style is similar to a Java class with a single superclass and additional interfaces.



Tip: You have seen the `CharSequence` interface in **Chapter 3**. Both `String` and `StringBuilder` (as well as a few more esoteric string-like classes) implement this interface. The interface contains methods that are common to all classes that manage sequences of characters. A common interface encourages programmers to write methods that use the `CharSequence` interface. Those methods work with instances of `String`, `StringBuilder`, and the other string-like classes.

Sadly, the `CharSequence` interface is rather paltry. You can get the length, iterate over the code points or code units, extract subsequences, and lexicographically compare two sequences. Java 17 adds an `isEmpty` method.

If you process strings, and those operations suffice for your tasks, accept `CharSequence` instances instead of strings.

6.1.4. Static and Private Methods

As of Java 8, you are allowed to add static methods to interfaces. There was never a technical reason why this should be outlawed. It simply seemed to be against the spirit of interfaces as abstract specifications.

Previously, it had been common to place static methods in companion classes. In the standard library, you'll find pairs of interfaces and utility classes such as `Collection`/`Collections`.

As an example, you can construct a path to a file or directory from a URI, or from a sequence of strings, using static methods in the `Path` interface:

```
public interface Path {
    public static Path of(URI uri) { . . . }
    public static Path of(String first, String... more) { . . . }
    . . .
}
```

In previous versions of Java, there was a separate `Paths` class to hold these methods. Nowadays, there is no longer a reason to provide a separate companion class for utility methods.

Methods in an interface can be `private`. A `private` method can be `static` or an instance method. Since private methods can only be used in the methods of the interface itself, their use is limited to being helper methods for the other methods of the interface.

6.1.5. Default Methods

You can supply a *default* implementation for any interface method. You must tag such a method with the `default` modifier.

```
public interface Comparable<T> {
    default int compareTo(T other) { return 0; }
        // by default, all elements are the same
}
```

Of course, that is not very useful since every realistic implementation of `Comparable` would override this method. But there are other situations where default methods can be useful. For example, in **Chapter 9** you will see an `Iterator` interface for visiting elements in a data structure. It declares a `remove` method as follows:

```
public interface Iterator<E> {
    boolean hasNext();
    E next();
    default void remove() { throw new UnsupportedOperationException("remove"); }
    . . .
}
```

If you implement an iterator, you need to provide the `hasNext` and `next` methods. There are no defaults for these methods—they depend on the data structure that you are traversing. But if your iterator is read-only, you don't have to worry about the `remove` method.

A default method can call other methods. For example, a `Collection` interface can define a convenience method

```
public interface Collection {
    int size(); // an abstract method
    default boolean isEmpty() { return size() == 0; }
    . . .
}
```

Then a programmer implementing `Collection` doesn't have to worry about implementing an `isEmpty` method.



Note: The `Collection` interface in the Java API does not actually do this. Instead, there is a class `AbstractCollection` that implements `Collection` and defines `isEmpty` in terms of `size`. Implementors of a collection are advised to extend `AbstractCollection`. That technique is obsolete. Just implement the methods in the interface.

An important use for default methods is *interface evolution*. Consider, for example, the `Collection` interface that has been a part of Java for many years. Suppose that a long time ago, you provided a class

```
public class Bag implements Collection
```

Later, in Java 8, a `stream` method was added to the interface.

Suppose the `stream` method was not a default method. Then the `Bag` class would no longer compile since it doesn't implement the new method. Adding a nondefault method to an interface is not *source-compatible*.

But suppose you don't recompile the class and simply use an old JAR file containing it. The class will still load, even with the missing method. Programs can still construct `Bag` instances, and nothing bad will happen. (Adding a method to an interface is *binary compatible*.) However, if a program calls the `stream` method on a `Bag` instance, an `AbstractMethodError` occurs.

Making the method a `default` method solves both problems. The `Bag` class will again compile. And if the class is loaded without being recompiled and the `stream` method is invoked on a `Bag` instance, the `Collection.stream` method is called.

6.1.6. Resolving Default Method Conflicts

What happens if the exact same method is defined as a default method in one interface and then again as a method of a superclass or another interface? Languages such as Scala and C++ have complex rules for resolving such ambiguities. Fortunately, the rules in Java are much simpler. Here they are:

1. Superclasses win. If a superclass provides a concrete method, default methods with the same name and parameter types are simply ignored.
2. Interfaces clash. If an interface provides a default method, and another interface contains a method with the same name and parameter types (default or not), then you must resolve the conflict by overriding that method.

Let's look at the second rule. Consider two interfaces with a `getName` method:

```
interface Person {
    default String getName() { return ""; }
}

interface Named {
    default String getName() { return getClass().getName() + "_" + hashCode(); }
}
```

What happens if you form a class that implements both of them?

```
class Student implements Person, Named { . . . }
```

The class inherits two inconsistent `getName` methods provided by the `Person` and `Named` interfaces. Instead of choosing one over the other, the Java compiler reports an error and leaves it up to the programmer to resolve the ambiguity. Simply provide a `getName` method in the `Student` class. In that method, you can choose one of the two conflicting methods, like this:

```
class Student implements Person, Named {
    public String getName() { return Person.super.getName(); }
    . . .
}
```

Now assume that the `Named` interface does not provide a default implementation for `getName`:

```
interface Named {
    String getName();
}
```

Can the `Student` class inherit the default method from the `Person` interface? This might be reasonable, but the Java designers decided in favor of uniformity. It doesn't matter how two interfaces conflict. If at least one interface provides an implementation, the compiler reports an error, and the programmer must resolve the ambiguity.

If neither interface provides a default for a shared method, then there is no conflict. An implementing class has two choices: implement the method, or leave it unimplemented. In the latter case, the class is itself abstract.

We just discussed name clashes between two interfaces. Now consider a class that extends a superclass and implements an interface, inheriting the same method from both. For example, suppose that `Person` is a class and `Student` is defined as

```
class Student extends Person implements Named { . . . }
```

In that case, only the superclass method matters, and any default method from the interface is simply ignored. In our example, `Student` inherits the `getName` method from `Person`, and it doesn't make any difference whether the `Named` interface provides a default for `getName` or not. This is the "class wins" rule.

The "class wins" rule ensures compatibility with old versions of Java. If you add default methods to an interface, it has no effect on code that worked before there were default methods.

Caution: You can never make a default method that redefines one of the methods in the `Object` class. For example, you can't define a default method for `toString` or `equals`, even though that might be attractive for interfaces such as `List`. As a consequence of the "class wins" rule, such a method could never win against `Object.toString` or `Object.equals`.

6.1.7. Interfaces and Callbacks

A common pattern in programming is the *callback* pattern. In this pattern, you specify the action that should occur whenever a particular event happens. For example, you may want a particular action to occur when a button is clicked or a menu item is selected. However, as you have not yet seen how to implement user interfaces, we will consider a similar but simpler situation.

The `javax.swing` package contains a `Timer` class that is useful if you want to be notified whenever a time interval has elapsed. For example, if a part of your

program contains a clock, you can ask to be notified every second so that you can update the clock face.

When you construct a timer, you set the time interval and tell it what it should do whenever the time interval has elapsed.

How do you tell the timer what it should do? In many programming languages, you supply the name of a function that the timer should call periodically. However, the classes in the Java standard library take an object-oriented approach. You pass an object of some class. The timer then calls one of the methods on that object. Passing an object is more flexible than passing a function because the object can carry additional information.

Of course, the timer needs to know what method to call. The timer requires that you specify an object of a class that implements the `ActionListener` interface of the `java.awt.event` package. Here is that interface:

```
public interface ActionListener {
    void actionPerformed(ActionEvent event);
}
```

The timer calls the `actionPerformed` method when the time interval has expired.

Suppose you want to print a message “At the tone, the time is . . .”, followed by a beep, once every second. You would define a class that implements the `ActionListener` interface. You would then place whatever statements you want to have executed inside the `actionPerformed` method.

```
class TimePrinter implements ActionListener {
    public void actionPerformed(ActionEvent event) {
        IO.println("At the tone, the time is "
            + Instant.ofEpochMilli(event.getWhen()));
        Toolkit.getDefaultToolkit().beep();
    }
}
```

Note the `ActionEvent` parameter of the `actionPerformed` method. This parameter gives information about the event, such as the time when the event happened. The call `event.getWhen()` returns the event time, measured in milliseconds since the “epoch” (January 1, 1970). By passing it to the static `Instant.ofEpochMilli` method, we get a more readable description.

Next, construct an object of this class and pass it to the `Timer` constructor.

```
var listener = new TimePrinter();
Timer t = new Timer(1000, listener);
```

The first argument of the `Timer` constructor is the time interval that must elapse between notifications, measured in milliseconds. We want to be notified every second. The second argument is the listener object.

Finally, start the timer.

```
t.start();
```

Every second, a message like

```
At the tone, the time is 2025-12-16T05:01:49.550Z
```

is displayed, followed by a beep.



Caution: Be sure to import `javax.swing.Timer`. There is also a `java.util.Timer` class that is slightly different.

Listing 6.3 puts the timer and its action listener to work. After the timer is started, the program puts up a message dialog and waits for the user to click the OK button to stop. While the program waits for the user, the current time is displayed every second. (If you omitted the dialog, the program would terminate as soon as the `main` method exits.)

Listing 6.3 v1ch06/TimerDemo.java

```
1  package v1ch06;
2
3  import module java.base;
4  import module java.desktop;
5
6  import javax.swing.Timer;
7
8  /**
9   * This program demonstrates how to use a Timer.
10  */
11 class TimerDemo {
12     void main() {
13         var listener = new TimePrinter();
14
15         // construct a timer that calls the listener once every second
16         var timer = new Timer(1000, listener);
17         timer.start();
18
19         // keep program running until the user selects "OK"
20         JOptionPane.showMessageDialog(null, "Quit program?");
21         System.exit(0);
22     }
23
24 }
25
```

```
class TimePrinter implements ActionListener {
    public void actionPerformed(ActionEvent event) {
        IO.println("At the tone, the time is " + Instant.ofEpochMilli(event.getWhen
        Toolkit.getDefaultToolkit().beep();
    }
}
```

`javax.swing.JOptionPane` 1.2

- `static void showMessageDialog(Component parent, Object message)`
  displays a dialog box with a message prompt and an OK button. The dialog is centered over the `parent` component. If `parent` is `null`, the dialog is centered on the screen.

`javax.swing.Timer` 1.2

- `Timer(int interval, ActionListener listener)`
  constructs a timer that notifies `listener` whenever `interval` milliseconds have elapsed.
- `void start()`
  starts the timer. Once started, the timer calls `actionPerformed` on its listeners.
- `void stop()`
  stops the timer. Once stopped, the timer no longer calls `actionPerformed` on its listeners.

`java.awt.Toolkit` 1.0

- `static Toolkit getDefaultToolkit()`
  gets the default toolkit. A toolkit contains information about the GUI environment.
- `void beep()`
  emits a beep sound.

6.1.8. The `Comparator` Interface

In **Section 6.1.1**, you have seen how you can sort an array of objects, provided they are instances of classes that implement the `Comparable` interface. For example, you can sort an array of strings since the `String` class implements `Comparable<String>`, and the `String.compareTo` method compares strings in dictionary order.

Now suppose we want to sort strings by increasing length, not in dictionary order. We can't have the `String` class implement the `compareTo` method in two ways —and at any rate, the `String` class isn't ours to modify.

To deal with this situation, there is a second version of the `Arrays.sort` method whose parameters are an array and a *comparator*—an instance of a class that implements the `Comparator` interface.

```
public interface Comparator<T> {
    int compare(T first, T second);
}
```

To compare strings by length, define a class that implements `Comparator<String>`:

```
class LengthComparator implements Comparator<String> {
    public int compare(String first, String second) {
        return first.length() - second.length();
    }
}
```

To actually do the comparison, you need to make an instance:

```
var comp = new LengthComparator();
if (comp.compare(words[i], words[j]) > 0) . . .
```

Contrast this call with `words[i].compareTo(words[j])`. The `compare` method is called on the comparator object, not the string itself.



Note: Even though the `LengthComparator` object has no state, you still need to make an instance of it. You need the instance to call the `compare` method—it is not a static method.

To sort an array, pass a `LengthComparator` object to the `Arrays.sort` method:

```
String[] friends = { "Peter", "Paul", "Mary" };
Arrays.sort(friends, new LengthComparator());
```

Now the array is either `["Paul", "Mary", "Peter"]` or `["Mary", "Paul", "Peter"]`.

You will see in **Section 6.2** how to use a `Comparator` much more easily with a lambda expression.



Note: The `String` class provides a `Comparator` for case-insensitive comparison. Here is how you can use it:

```
Arrays.sort(friends, String.CASE_INSENSITIVE_ORDER);
```



Caution: Do not try to shuffle an array by sorting it with a comparator that randomly returns positive or negative integers.

There are three rules that a comparator needs to fulfill:

1. Reflexivity: When `x` and `y` are equal, the comparator yields 0.
2. Antisymmetry: When swapping the arguments of the comparator, the sign of the result is swapped.
3. Transitivity: When `x` comes before `y` and `y` comes before `z`, then `x` must come before `z`.

The algorithm that `Arrays.sort` uses (called "Timsort") doesn't check these rules for all elements, but it can sometimes detect a rule violation at a trivial cost. Then it throws an exception with the message "Comparison method violates its general contract!". With an array of 1,000 elements, the chance of this occurring with a random comparator is over 10%.

The `Collections.shuffle` method randomly shuffles a list. To shuffle an array, first turn it into a list and then shuffle that.

6.1.9. Object Cloning

In this section, we discuss the `Cloneable` interface that indicates that a class has provided a safe `clone` method. Since cloning is not all that common, and the details are quite technical, you may just want to glance at this material until you need it.

To understand what cloning means, recall what happens when you make a copy of a variable holding an object reference. The original and the copy are references to the same object (see **Figure 6.1**). This means a change to either variable also affects the other.

```
var original = new Employee("John Public", 50000);
Employee copy = original;
copy.raiseSalary(10); // oops--also changed original
```

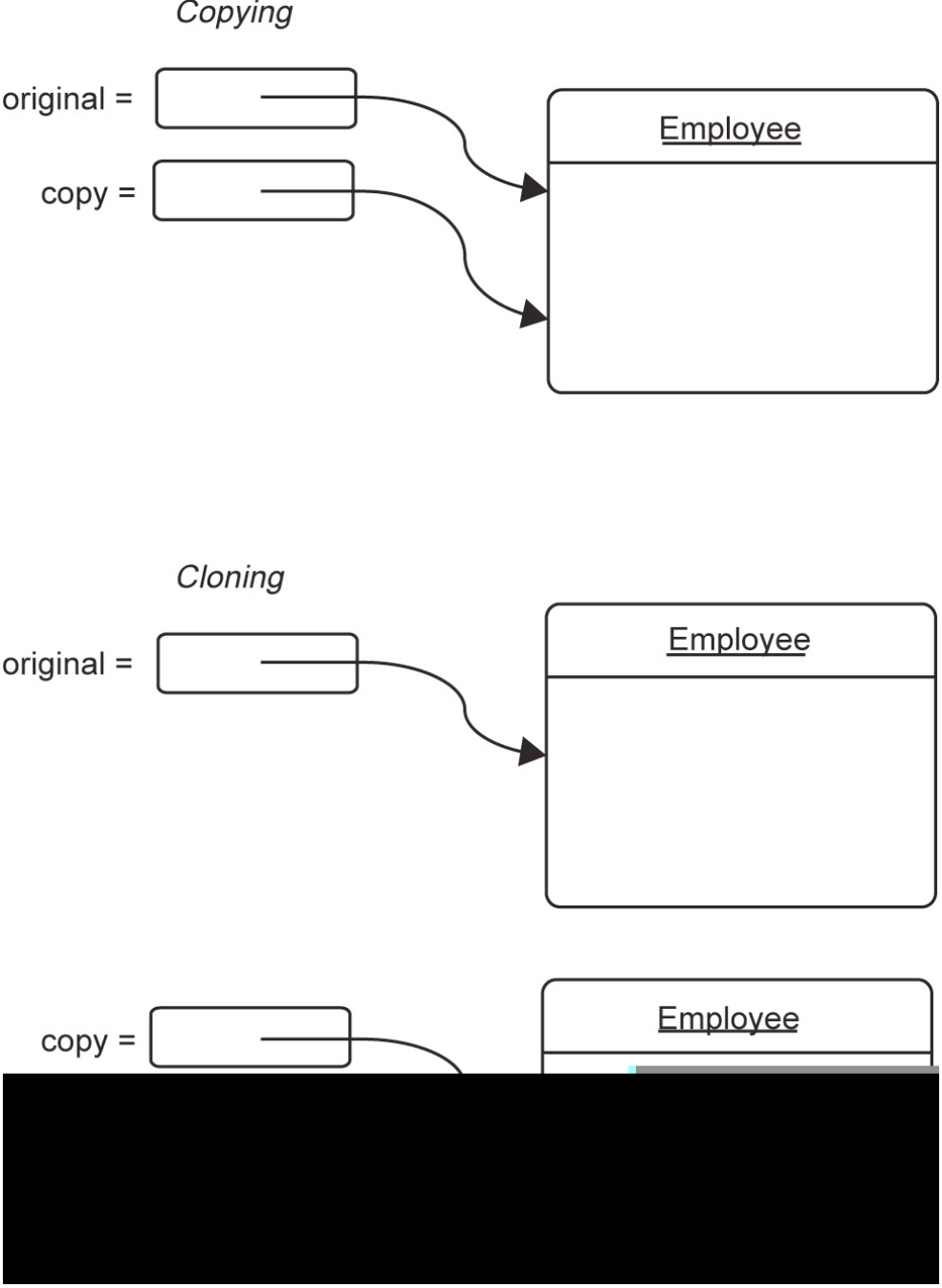


Figure 6.1: Copying and cloning

If you would like `copy` to be a new object that begins its life being identical to `original` but whose state can diverge over time, use the `clone` method.

```
Employee copy = original.clone();
copy.raiseSalary(10); // OK--original unchanged
```

But it isn't quite so simple. The `clone` method is a `protected` method of `Object`, which means that your code cannot simply call it. Only the `Employee` class can clone `Employee` objects. There is a reason for this restriction. Think about the way in which the `Object` class can implement `clone`. It knows nothing about the object at all, so it can make only a field-by-field copy. If all instance fields in the object are numbers or other basic types, copying the fields is just fine. But if the object contains references to subobjects, then copying the field gives

you another reference to the same subobject, so the original and the cloned objects still share some information.

To visualize that, consider the `Employee` class that was introduced in **Chapter 4**. **Figure 6.2** shows what happens when you use the `clone` method of the `Object` class to clone such an `Employee` object. As you can see, the default cloning operation is "shallow"—it doesn't clone objects that are referenced inside other objects. (The figure shows a shared `Date` object. For reasons that will become clear shortly, this example uses a version of the `Employee` class in which the hire day is represented as a `Date`.)

Figure 6.2: A shallow copy

Does it matter if the copy is shallow? It depends. If the subobject shared between the original and the shallow clone is *immutable*, then the sharing is safe. This certainly happens if the subobject belongs to an immutable class, such as `String`. Alternatively, the subobject may simply remain constant throughout the lifetime of the object, with no mutators touching it and no methods yielding a reference to it.

Quite frequently, however, subobjects are mutable, and you must redefine the `clone` method to make a *deep copy* that clones the subobjects as well. In our example, the `hireDay` field is a `Date`, which is mutable, so it too must be cloned. (For that reason, this example uses a field of type `Date`, not `LocalDate`, to demonstrate the cloning process. Had `hireDay` been an instance of the immutable `LocalDate` class, no further action would have been required.)

For every class, you need to decide whether

1. The default `clone` method is good enough;
2. The default `clone` method can be patched up by calling `clone` on the mutable subobjects; or
3. `clone` should not be attempted.

The third option is actually the default. To choose either the first or the second option, a class must

1. Implement the `Cloneable` interface; and
2. Redefine the `clone` method with the `public` access modifier.



Note: The `clone` method is declared `protected` in the `Object` class, so that your code can't simply call `anObject.clone()`. But aren't protected methods accessible from any subclass, and isn't every class a subclass of `Object`? Fortunately, the rules for protected access are more subtle (see **Chapter 5**). A subclass can call a protected `clone` method only to clone *its own* objects. You must redefine `clone` to be public to allow objects to be cloned by any method.

In this case, the appearance of the `Cloneable` interface has nothing to do with the normal use of interfaces. In particular, it does *not* specify the `clone` method —that method is inherited from the `Object` class. The interface merely serves as a tag, indicating that the class designer understands the cloning process. Objects are so paranoid about cloning that they generate a checked exception if an object requests cloning but does not implement that interface.



Note: The `Cloneable` interface is one of a handful of *tagging interfaces* that Java provides. (Some programmers call them *marker interfaces*.) Recall that the usual purpose of an interface such as `Comparable` is to ensure that a class implements a particular method or set of methods. A tagging interface has no methods; its only purpose is to allow the use of `instanceof` in a type inquiry:

```
if (obj instanceof Cloneable) . . .
```

I recommend that you do not use tagging interfaces in your own programs.

Even if the default (shallow copy) implementation of `clone` is adequate, you still need to implement the `Cloneable` interface, redefine `clone` to be public, and call `super.clone()`. Here is an example:

```
class Employee implements Cloneable {
    // public access, change return type
    public Employee clone() throws CloneNotSupportedException {
        return (Employee) super.clone();
    }
    . . .
}
```



Note: Note that in the `Object` class, the `clone` method has return type `Object`. In a subclass, you can specify the correct return type for your `clone` methods. This is an example of covariant return types (see **Chapter 5**).

The `clone` method that you just saw adds no functionality to the shallow copy provided by `Object.clone`. It merely makes the method public. To make a deep copy, you have to work harder and clone the mutable instance fields.

Here is an example of a `clone` method that creates a deep copy:

```
class Employee implements Cloneable {
    . . .
    public Employee clone() throws CloneNotSupportedException {
        // call Object.clone()
        Employee cloned = (Employee) super.clone();

        // clone mutable fields
        cloned.hireDay = (Date) hireDay.clone();
        return cloned;
    }
}
```

The `clone` method of the `Object` class threatens to throw a `CloneNotSupportedException`—it does that whenever `clone` is invoked on an object whose class does not implement the `Cloneable` interface. Of course, the `Employee` and `Date` classes implement the `Cloneable` interface, so the exception won't be thrown. However, the compiler does not know that. Therefore, we declared the exception:

```
public Employee clone() throws CloneNotSupportedException
```



Note: Would it be better to catch the exception instead? (See **Chapter 7** for details on catching exceptions.)

```
public Employee clone() {
    try {
        Employee cloned = (Employee) super.clone();
        . . .
    }
    catch (CloneNotSupportedException e) { return null; }
    // this won't happen, since we are Cloneable
}
```

This is appropriate for `final` classes. Otherwise, it is better to leave the `throws` specifier in place. That gives subclasses the option of throwing a `CloneNotSupportedException` if they can't support cloning.

You have to be careful about cloning of subclasses. For example, once you have defined the `clone` method for the `Employee` class, anyone can use it to clone `Manager` objects. Can the `Employee` clone method do the job? It depends on the fields of the `Manager` class. In our case, there is no problem because the `bonus` field has primitive type. But `Manager` might have acquired fields that require a

deep copy or are not cloneable. There is no guarantee that the implementor of the subclass has fixed `clone` to do the right thing. For that reason, the `clone` method is declared as `protected` in the `Object` class. But you don't have that luxury if you want the users of your classes to invoke `clone`.

Should you implement `clone` in your own classes? If your clients need to make deep copies, then you probably should. Some authors feel that you should avoid `clone` altogether and instead implement another method for the same purpose. I agree that `clone` is rather awkward, but you'll run into the same issues if you shift the responsibility to another method. At any rate, cloning is less common than you may think. Less than five percent of the classes in the standard library implement `clone`.

The program in **Listing 6.4** clones an instance of the class `Employee` (**Listing 6.5**), then invokes two mutators. The `raiseSalary` method changes the value of the `salary` field, whereas the `setHireDay` method changes the state of the `hireDay` field. Neither mutation affects the original object because `clone` has been defined to make a deep copy.



Note: All array types have a `clone` method that is public, not protected. You can use it to make a new array that contains copies of all elements. For example:

```
int[] luckyNumbers = { 2, 3, 5, 7, 11, 13 };
int[] cloned = luckyNumbers.clone();
cloned[5] = 12; // doesn't change luckyNumbers[5]
```

Listing 6.4 v1ch06/clone/CloneDemo.java

```
1  package v1ch06.clone;
2
3  /**
4   * This program demonstrates cloning.
5   */
6  class CloneDemo {
7      void main() throws Exception {
8          var original = new Employee("John Q. Public", 50000);
9          original.setHireDay(2000, 1, 1);
10         Employee copy = original.clone();
11         copy.raiseSalary(10);
12         copy.setHireDay(2002, 12, 31);
13         IO.println("original=" + original);
14         IO.println("copy=" + copy);
15     }
16 }
```

Listing 6.5 v1ch06/clone/Employee.java

```
package v1ch06.clone;

import module java.base;

/**
 * The familiar Employee class, modified to be cloneable, and with hireDay of type
 */
public class Employee implements Cloneable {
    private String name;
    private double salary;
    private Date hireDay;

    public Employee(String name, double salary) {
        this.name = name;
        this.salary = salary;
        hireDay = new Date();
    }

    public Employee clone() throws CloneNotSupportedException {
        // call Object.clone()
        Employee cloned = (Employee) super.clone();

        // clone mutable fields
        cloned.hireDay = (Date) hireDay.clone();

        return cloned;
    }

    /**
     * Set the hire day to a given date.
     * @param year the year of the hire day
     * @param month the month of the hire day
     * @param day the day of the hire day
     */
    public void setHireDay(int year, int month, int day) {
        long epochMillis = LocalDate.of(year, month, day).atStartOfDay(ZoneId.syste
                .toEpochSecond() * 1000;

        // example of instance field mutation
        hireDay.setTime(epochMillis);
    }

    public void raiseSalary(double byPercent) {
        double raise = salary * byPercent / 100;
        salary += raise;
    }

    public String toString() {
        return "Employee[name=" + name + ",salary=" + salary + ",hireDay=" + hireDa
    }
}
```

6.2. Lambda Expressions

In the following sections, you will learn how to use lambda expressions for defining blocks of code with a concise syntax, and how to write code that consumes lambda expressions.

6.2.1. Why Lambdas?

A lambda expression is a block of code that you can pass around so it can be executed later, once or multiple times. Before getting into the syntax (or even the curious name), let's step back and observe where we have used such code blocks in Java.

In **Section 6.1.7**, you saw how to do work in timed intervals. Put the work into the `actionPerformed` method of an `ActionListener`:

```
class Worker implements ActionListener {
    public void actionPerformed(ActionEvent event) {
        // do some work
    }
}
```

Then, when you want to repeatedly execute this code, you construct an instance of the `Worker` class. You then submit the instance to a `Timer` object.

The key point is that the `actionPerformed` method contains code that you want to execute later.

Or consider sorting with a custom comparator. If you want to sort strings by length instead of the default dictionary order, you can pass a `Comparator` object to the `sort` method:

```
class LengthComparator implements Comparator<String> {
    public int compare(String first, String second) {
        return first.length() - second.length();
    }
}
. . .
Arrays.sort(strings, new LengthComparator());
```

The `compare` method isn't called right away. Instead, the `sort` method keeps calling the `compare` method, rearranging the elements if they are out of order, until the array is sorted. You give the `sort` method a snippet of code needed to compare elements, and that code is integrated into the rest of the sorting logic, which you'd probably not care to reimplement.

Both examples have something in common. A block of code was passed to someone—a timer, or a `sort` method. That code block was called at some later time.

But giving someone a block of code doesn't seem easy. Java is an object-oriented language, so we constructed an object belonging to a class that has a method with the desired code.

In other languages, it is possible to work with blocks of code directly. The Java designers have resisted adding this feature for a long time. After all, a great strength of Java is its simplicity and consistency. A language can become an unmaintainable mess if it includes every feature that yields marginally more concise code. However, in those other languages it isn't just easier to spawn a thread or to register a button click handler; large swaths of their APIs are simpler, more consistent, and more powerful. In Java, one could have written similar APIs taking objects of classes that implement a particular interface, but such APIs would be unpleasant to use.

For some time, the question was not whether to augment Java for functional programming, but how to do it. It took several years of experimentation before a design emerged that is a good fit for Java. In the next section, you will see how you can work with blocks of code in Java.

6.2.2. The Syntax of Lambda Expressions

Consider again the sorting example from the preceding section. We pass code that checks whether one string is shorter than another. We compute

```
first.length() - second.length()
```

What are `first` and `second`? They are both strings. Java is a strongly typed language, and we must specify that as well:

```
(String first, String second) ->
    first.length() - second.length()
```

You have just seen your first *lambda expression*. Such an expression is simply a block of code, together with the specification of any variables that must be passed to the code.

Why the name? Many years ago, before there were any computers, the logician Alonzo Church wanted to formalize what it means for a mathematical function to be effectively computable. (Curiously, there are functions that are known to exist, but nobody knows how to compute their values.) He used the Greek letter lambda (λ) to mark parameters. Had he known about the Java API, he would have written

```
λfirst.λsecond.first.length() - second.length()
```



Note: Why the letter λ? Did Church run out of other letters of the alphabet? Actually, the venerable *Principia Mathematica* used the ^ accent to denote

> free variables, which inspired Church to use an uppercase lambda Λ for parameters. But in the end, he switched to the lowercase version. Ever since, an expression with parameter variables has been called a lambda expression.

What you have just seen is a simple form of lambda expressions in Java: parameters, the `->` arrow, and an expression. If the code carries out a computation that doesn't fit in a single expression, write it exactly like you would have written a method: enclosed in `{}` and with explicit `return` statements. For example,

```
(String first, String second) -> {
    if (first.length() < second.length()) return -1;
    else if (first.length() > second.length()) return 1;
    else return 0;
}
```

If a lambda expression has no parameters, you still supply empty parentheses, just as with a parameterless method:

```
() -> { return 1 + (int)(Math.random() * 6); }
```

If the parameter types of a lambda expression can be inferred, you can omit them. For example,

```
Comparator<String> comp =
    (first, second) // same as (String first, String second)
        -> first.length() - second.length();
```

Here, the compiler can deduce that `first` and `second` must be strings because the lambda expression is assigned to a string comparator. (We will have a closer look at this assignment in the next section.)

If a method has a single parameter with inferred type, you can even omit the parentheses:

```
ActionListener listener = event ->
    IO.println("The time is "
        + Instant.ofEpochMilli(event.getWhen()));
            // instead of (event) -> . . . or (ActionEvent event) -> . . .
```

If a parameter of a lambda expression is never used, you can denote it with an underscore:

```
ActionListener listener = _ ->
    IO.println("The action occurred at " + Instant.now());
Comparator<String> comp = (_, _) -> 0;
```



Note: You can also use `var` to denote an inferred parameter type. This isn't common. The syntax was invented for attaching annotations (see **Chapter 11**):

```
(@NonNull var first, @NonNull var second) -> first.length() - second.length()
```

You never specify the result type of a lambda expression. It is always inferred from context. For example, the expression

```
(String first, String second) -> first.length() - second.length()
```

can be used in any context where a result of type `int` is expected.



Note: It is illegal for a lambda expression to return a value in some branches but not in others. For example, `(int x) -> { if (x >= 0) return 1; }` is invalid.

The program in **Listing 6.6** shows how to use lambda expressions for a comparator and an action listener.

Listing 6.6 v1ch06/LambdaDemo.java

```
1  package v1ch06;
2
3  import module java.base;
4  import module java.desktop;
5
6  import javax.swing.Timer;
7
8  /**
9   * This program demonstrates the use of lambda expressions.
10  */
11 class LambdaDemo {
12     void main() {
13         var planets = new String[]{"Mercury", "Venus", "Earth", "Mars", "Jupiter",
14                 "Uranus", "Neptune"};
15         IO.println(Arrays.toString(planets));
16         IO.println("Sorted in dictionary order:");
17         Arrays.sort(planets);
18         IO.println(Arrays.toString(planets));
19         IO.println("Sorted by length:");
20         Arrays.sort(planets, (first, second) -> first.length() - second.length());
21         IO.println(Arrays.toString(planets));
22
23         var timer = new Timer(1000, _ -> IO.println("The time is " + Instant.now()));
24         timer.start();
25
26         // keep program running until user selects "OK"
27         JOptionPane.showMessageDialog(null, "Quit program?");
```

```
28          System.exit(0);
29      }
30  }
```

6.2.3. Functional Interfaces

As we discussed, there are many existing interfaces in Java that encapsulate blocks of code, such as `ActionListener` or `Comparator`. Lambdas are compatible with these interfaces.

You can supply a lambda expression whenever an object of an interface with a single abstract method is expected. Such an interface is called a *functional interface*.



Note: You may wonder why a functional interface must have a single *abstract* method. Aren't all methods in an interface abstract? Actually, it has always been possible for an interface to redeclare methods from the `Object` class such as `toString` or `clone`, and these declarations do not make the methods abstract. (Some interfaces in the Java API redeclare `Object` methods in order to attach javadoc comments. Check out the `Comparator` API for an example.) More importantly, as you saw in **Section 6.1.5**, interfaces can declare nonabstract methods.

To demonstrate the conversion to a functional interface, consider the `Arrays.sort` method. Its second parameter requires an instance of `Comparator`, an interface with a single method. Simply supply a lambda:

```
Arrays.sort(words,
    (first, second) -> first.length() - second.length());
```

Behind the scenes, the `Arrays.sort` method receives an object of some class that implements `Comparator<String>`. Invoking the `compare` method on that object executes the body of the lambda expression. The management of these objects and classes is completely implementation-dependent, and it can be much more efficient than using traditional inner classes. It is best to think of a lambda expression as a function, not an object, and to accept that it can be passed to a functional interface.

This conversion to interfaces is what makes lambda expressions so compelling. The syntax is short and simple. Here is another example:

```
var timer = new Timer(1000, event -> {
    IO.println("At the tone, the time is "
        + Instant.ofEpochMilli(event.getWhen()));
    Toolkit.getDefaultToolkit().beep();
});
```

That's a lot easier to read than the alternative with a class that implements the `ActionListener` interface.

In fact, conversion to a functional interface is the *only* thing that you can do with a lambda expression in Java. In other programming languages that support function literals, you can declare function types such as `(String, String) -> int`, declare variables of those types, and use the variables to save function expressions. However, the Java designers decided to stick with the familiar concept of interfaces instead of adding function types to the language.



Note: You can't even assign a lambda expression to a variable of type `Object`—`Object` is not a functional interface.

6.2.4. Function Types

The Java API defines a number of very generic functional interfaces in the `java.util.function` package. One of the interfaces, `BiFunction<T, U, R>`, describes functions with parameter types `T` and `U` and return type `R`. You can save your string comparison lambda in a variable of that type:

```
BiFunction<String, String, Integer> comp =
    (first, second) -> first.length() - second.length();
```

Note that this interface does not help you with sorting. There is no `Arrays.sort` method that wants a `BiFunction`. If you have used a functional programming language before, you may find this curious. But for Java programmers, it's pretty natural. An interface such as `Comparator` has a specific purpose, not just a method with given parameter and return types. When you want to do something with lambda expressions, you still want to keep the purpose of the expression in mind, and have a specific functional interface for it.

A particularly useful interface in the `java.util.function` package is `Predicate`:

```
public interface Predicate<T> {
    boolean test(T t);
    // additional default and static methods
}
```

The `ArrayList` class has a `removeIf` method whose parameter is a `Predicate`. It is specifically designed to pass a lambda expression. For example, the following statement removes all `null` values from an array list:

```
list.removeIf(e -> e == null);
```

Another useful functional interface is `Supplier<T>`:

```
public interface Supplier<T> {
    T get();
}
```

A supplier has no parameters and yields a value of type `T` when the `get` method is called:

```
Supplier<Integer> die = () -> (int)(Math.random() * 6) + 1;
int outcome = die.get();
```

Suppliers are used for *lazy evaluation*. For example, consider the call

```
LocalDate hireDay = Objects.requireNonNullElse(day,
    LocalDate.of(1970, 1, 1));
```

This is not optimal. We expect that `day` is rarely `null`, so we only want to construct the default `LocalDate` when necessary. By using the supplier, we can defer the computation:

```
LocalDate hireDay = Objects.requireNonNullElseGet(day,
    () -> LocalDate.of(1970, 1, 1));
```

The `requireNonNullElseGet` method only calls the supplier when the value is needed.

Functional interfaces that involve primitive types are a little cumbersone. Consider a function consuming an `int` and yielding an object of type `T`. You could use a `Function<Integer, T>`, but then the argument must be boxed in each call. Instead, there is a functional interface `IntFunction<T>`. Conversely, if a function has a return value of type `int`, the `ToIntFunction<T>` interface is more efficient than `Function<T, Integer>`. Finally, if both argument and return value are `int`, there is an `IntUnaryOperator` interface.

As the user of an API, you don't usually care about this subtlety. Consider the `Arrays.setAll` method. It sets all values of an array to the result of a function whose argument is the array index. Here, we set all elements to the square of the index:

```
var values = new int[100];
Arrays.setAll(values, i -> i * i); // [0, 1, 4, 9, 16, . . ., 9801]
```

There are overloaded versions of `setAll` for arrays of type `int[]`, `long[]`, `double[]`, and a generic array `T[]`. Here, the `int[]` overload has as second parameter an `IntUnaryOperator`. But as the user of the method, you don't care.

You just supply the lambda expression, which you can do without worrying about the difference between primitive types and their wrapper classes.



Caution: It is nice that a lambda expression can match primitive and wrapper types in a functional interface. But it is an error if both matches could occur. Consider a utility class that provides these methods:

```
public static int[] fill(int n, IntUnaryOperator op)
public static Object[] fill(int n, IntFunction<Object> op)
```

A call `fill(n, i -> i * i)` will not compile since it is ambiguous.

You can catch such problems in your API by compiling with the `-Xlint:overloads` option.

6.2.5. Method References

Sometimes, a lambda expression involves a single method. For example, suppose you simply want to print the event object whenever a timer event occurs. Of course, you could call

```
var timer = new Timer(1000, event -> IO.println(event));
```

It would be nicer if you could just pass the `println` method to the `Timer` constructor. Here is how you do that:

```
var timer = new Timer(1000, IO::println);
```

The expression `IO::println` is a *method reference*. It directs the compiler to produce an instance of a functional interface, overriding the single abstract method of the interface to call the given method. In this example, an `ActionListener` is produced whose `actionPerformed(ActionEvent e)` method calls `IO.println(e)`.



Note: Like a lambda expression, a method reference is not an object. It gives rise to an object when assigned to a variable whose type is a functional interface.



Note: There are two overloaded `println` methods in the `IO` class, one with a parameter of type `Object`, and another with no parameters. The compiler needs to figure out which one to use, depending on context. In our example, the method reference `IO::println` must be turned into an `ActionListener` instance with a method

```
void actionPerformed(ActionEvent e)
```

The `println(Object obj)` method is selected from the overloaded `println` methods. When the `actionPerformed` method is called, the event object is printed.

Now suppose we assign the same method reference to a different functional interface:

```
Runnable task = IO::println;
```

The `Runnable` functional interface has a single abstract method with no parameters

```
void run()
```

In this case, the `println()` method with no parameters is chosen. Calling `task.run()` prints a blank line.

In this example, `IO::println` is a reference to a static method. The `::` notation has three variants:

1. `Class::staticMethod`
2. `Class::instanceMethod`
3. `object::instanceMethod`

Table 6.1 walks you through several examples.

Table 6.1: Method Reference Examples

Method Reference	Equivalent Lambda Expression	Notes
`separator::equals`	`x -> separator.equals(x)`	This is a method expression with an *object* and an instance method. The lambda parameter is passed as the explicit parameter of the method.
`String::strip`	`x -> x.strip()`	This is a method expression with a *class* and an instance method. The lambda parameter becomes the implicit parameter.

Method Reference	Equivalent Lambda Expression	Notes
`String::concat`	`(x, y) -> x.concat(y)`	Again, we have an instance method, but this time, with an explicit parameter. As before, the *first* lambda parameter becomes the implicit parameter, and the remaining ones are passed to the method.
`Integer::valueOf`	`x -> Integer.valueOf(x)`	This is a method expression with a *static* method. The lambda parameter is passed to the static method.
`Integer::sum`	`(x, y) -> Integer.sum(x, y)`	This is another static method, but this time with two parameters. Both lambda parameters are passed to the static method. The `Integer.sum` method was specifically created to be used as a method reference. As a lambda, you could just write `(x, y) -> x + y`.
`String::new`	`x -> new String(x)`	This is a constructor reference—see **Section 6.2.6**. The lambda parameters are passed to the constructor.
`String[]::new`	`n -> new String[n]`	This is an array constructor reference —see **Section 6.2.6**. The lambda parameter is the array length.

Note that a lambda expression can only be rewritten as a method reference if the body of the lambda expression calls a single method and doesn't do anything else. Consider the lambda expression

```
s -> s.length() == 0
```

There is a single method call. But there is also a comparison, so you can't use a method reference here.



Note: When there are multiple overloaded methods with the same name, the compiler will try to find from the context which one you mean. For example, there are two versions of the `Math.max` method, one for integers and one for `double` values. Which one gets picked depends on the method parameters of the functional interface to which `Math::max` is converted. Just like lambda expressions, method references don't live in isolation. They are always turned into instances of functional interfaces.



Note: Sometimes, the API contains methods that are specifically intended to be used as method references. For example, the `Objects` class has a method `isNull` to test whether an object reference is `null`. At first glance, this doesn't seem useful because the test `obj == null` is easier to read than `Objects.isNull(obj)`. But you can pass the method reference to any method with a `Predicate` parameter. For example, to remove all `null` references from a list, you can call

```
list.removeIf(Objects::isNull);
    // A bit easier to read than list.removeIf(e -> e == null);
```



Note: There is a tiny difference between a method reference with an object and its equivalent lambda expression. Consider a method reference such as `separator::equals`. If `separator` is `null`, forming `separator::equals` immediately throws a `NullPointerException`. The lambda expression `x -> separator.equals(x)` only throws a `NullPointerException` if it is invoked.

You can capture the `this` parameter in a method reference. For example, `this::equals` is the same as `x -> this.equals(x)`. It is also valid to use `super`. The method expression

```
super::instanceMethod
```

uses `this` as the target and invokes the superclass version of the given method. Here is an artificial example that shows the mechanics:

```
class Greeter {
    public void greet(ActionEvent event) {
        IO.println("Hello, the time is "
            + Instant.ofEpochMilli(event.getWhen()));
    }
}

class RepeatedGreeter extends Greeter {
    public void greet(ActionEvent event) {
        var timer = new Timer(1000, super::greet);
        timer.start();
    }
}
```

When the `RepeatedGreeter.greet` method starts, a `Timer` is constructed that executes the `super::greet` method on every timer tick.

6.2.6. Constructor References

Constructor references are just like method references, except that the name of the method is `new`. For example, `Person::new` is a reference to a `Person` constructor. Which constructor? It depends on the context. Suppose you have a list of strings. Then you can turn it into an array of `Person` objects, by calling the constructor on each of the strings, with the following invocation:

```
ArrayList<String> names = . . .;
Stream<Person> stream = names.stream().map(Person::new);
List<Person> people = stream.toList();
```

We will discuss the details of the `stream`, `map`, and `toList` methods in **Chapter 1 of Volume II**. For now, what's important is that the `map` method calls the `Person(String)` constructor for each list element. If there are multiple `Person` constructors, the compiler picks the one with a `String` parameter because it infers from the context that the constructor is called with a string.

You can form constructor references with array types. For example, `int[]::new` is a constructor reference with one parameter: the length of the array. It is equivalent to the lambda expression `n -> new int[n]`.

Array constructor references are useful to overcome a limitation of Java. As you will see in **Chapter 8**, it is not possible to construct an array of a generic type `T`. (The expression `new T[n]` is an error since it would be "erased" to `new Object[n]`). That is a problem for library authors. For example, suppose we want to have an array of `Person` objects. The `Stream` interface has a `toArray` method that returns an `Object` array:

```
Object[] people = stream.toArray();
```

But that is unsatisfactory. The user wants an array of references to `Person`, not references to `Object`. The stream library solves that problem with constructor references. Pass `Person[]::new` to the `toArray` method:

```
Person[] people = stream.toArray(Person[]::new);
```

The `toArray` method invokes this constructor to obtain an array of the correct type. Then it fills and returns the array.



Caution: Sometimes, it is surprising which overloaded variant is chosen when passing a method or constructor reference. Consider this code snippet:

```
var dates = new Date[100];
Arrays.setAll(dates, Date::new);
```

At first glance, it looks as if all elements would be set to the current date, by calling the no-argument constructor `new Date()` each time. But actually, the second parameter of `setAll` is an `IntFunction`, which receives the index of the element. Therefore, an entirely different constructor is invoked, `new Date(i)`, where `i` ranges from 0 to 99. That constructor sets the date to a given number of milliseconds from the "epoch," January 1, 1970.

6.2.7. Variable Scope

Often, you want to be able to access a variable from the enclosing method in a lambda expression. Consider this example:

```
public static void repeatMessage(String text, int delay) {
    ActionListener listener = event -> {
        IO.println(text);
    };
    new Timer(delay, listener).start();
}
```

Consider a call

```
repeatMessage("Hello", 1000); // prints Hello every 1,000 milliseconds
```

Now look at the variable `text` inside the lambda expression. Note that this variable is *not* defined in the lambda expression. Instead, it is a parameter variable of the `repeatMessage` method.

If you think about it, something nonobvious is going on here. The code of the lambda expression may run long after the call to `repeatMessage` has returned

and the parameter variables are gone. How does the `text` variable stay around?

To understand what is happening, we need to refine our understanding of a lambda expression. A lambda expression has three ingredients:

1. A block of code
2. Parameters
3. Values for the *free* variables—that is, the variables that are not parameters and not defined inside the code

In our example, the lambda expression has one free variable, `text`. The data structure representing the lambda expression must store the values for the free variables—in our case, the string `"Hello"`. We say that such values have been *captured* by the lambda expression. (It's an implementation detail how that is done. For example, one can translate a lambda expression into an object with a single method, so that the values of the free variables are copied into instance fields of that object.)



Note: The technical term for a block of code together with the values of the free variables is a *closure*. If someone gloats that their language has closures, rest assured that Java has them as well. In Java, lambda expressions are closures.

As you have seen, a lambda expression can capture the value of a variable in the enclosing scope. In Java, to ensure that the captured value is well-defined, there is an important restriction. In a lambda expression, you can only reference variables whose value doesn't change. For example, the following is illegal:

```
public static void countDown(int start, int delay) {
    ActionListener listener = _ -> {
        start--; // ERROR: Can't mutate captured variable
        IO.println(start);
        if (start > 0) countDown(start, delay);
    };
    new Timer(delay, listener).start();
}
```

There is a reason for this restriction. Mutating variables in a lambda expression is not safe when multiple actions are executed concurrently. This won't happen for the kinds of actions that we have seen so far, but in general, it is a serious problem. See **Chapter 10** for more information on this important issue.

It is also illegal to refer, in a lambda expression, to a variable that is mutated outside. For example, the following is illegal:

```
public static void multiMessage(String text, int count) {
    for (int i = 1; i <= count; i++) {
        ActionListener listener = _ -> IO.println(i + ": " + text);
            // ERROR: Cannot refer to changing i
```

```
            new Timer(1000, listener).start();
        }
    }
```

The rule is that any captured variable in a lambda expression must be *effectively final*. An effectively final variable is a variable that is never assigned a new value after it has been initialized. In our case, `text` always refers to the same `String` object, and it is OK to capture it. However, the value of `i` is mutated, and therefore `i` cannot be captured.

The body of a lambda expression has *the same scope as a nested block*. The same rules for name conflicts and shadowing apply. It is illegal to declare a parameter or a local variable in the lambda that has the same name as a local variable.

```
Path first = Path.of("/usr/bin");
Comparator<String> comp =
    (first, second) -> first.length() - second.length();
        // ERROR: Variable first already defined
```

Inside a method, you can't have two local variables with the same name, and therefore, you can't introduce such variables in a lambda expression either.

6.2.8. Lambda Expressions and `this`

A lambda expression does not have its own `this` reference.

When you use the `this` keyword in a lambda expression, it denotes the `this` parameter of the enclosing method. For example, consider

```
public class Greeter {
    public void greet(String text) {
        ActionListener listener = event -> {
            IO.println(this);
            IO.println(text);
        }
    }
    . . .
}
```

In the call `IO.println(this)`, the `Greeter` object, *not* the `ActionListener` instance, is printed.

There is nothing special about the use of `this` in a lambda expression. The scope of the lambda expression is nested inside the `greet` method, and `this` has the same meaning anywhere in that method.

So far, we have discussed what happens when a lambda expression accesses a variable from the enclosing method. It is also ok to access fields from the enclosing class.

```
class Greeter {
    private String text;
    public void delayMessage(int delay) {
        ActionListener listener = event -> IO.println(text);
            // Ok to access field
        new Timer(delay, listener).start();
    }
    public void greet() {
        text = "Hello";
        delayMessage(1000);
        text = "Goodbye";
        delayMessage(2000);
    }
}
```

Note that `text` is not final! It seems like there are different rules for local variables and fields. But actually, there is only one rule. In the lambda expression, `IO.println(text)` means `IO.println(this.text)`. The lambda expression accesses the final `this` value.

6.2.9. Processing Lambda Expressions

Up to now, you have seen how to produce lambda expressions and pass them to a method that expects a functional interface. Now let us see how to write methods that can consume lambda expressions.

The point of using lambdas is *deferred execution*. After all, if you wanted to execute some code right now, you'd do that, without wrapping it inside a lambda. There are many reasons for executing code later, such as:

- Running the code in a separate thread
- Running the code multiple times
- Running the code at the right point in an algorithm (for example, the comparison operation in sorting)
- Running the code when something happens (a button was clicked, data has arrived, and so on)
- Running the code only when necessary

Let's look at a simple example. Suppose you want to repeat an action `n` times. The action and the count are passed to a `repeat` method:

```
repeat(10, () -> IO.println("Hello, World!"));
```

To accept the lambda, we need to pick (or, in rare cases, provide) a functional interface. **Table 6.2** lists the most important functional interfaces that are provided in the Java API. In this case, we can use the `Runnable` interface:

```
public static void repeat(int n, Runnable action) {
    for (int i = 0; i < n; i++) action.run();
}
```

Note that the body of the lambda expression is executed when `action.run()` is called.

Now let's make this example a bit more sophisticated. We want to tell the action in which iteration it occurs. For that, we need to pick a functional interface that has a method with an `int` parameter and a `void` return. The standard interface for processing `int` values is

```
public interface IntConsumer {
    void accept(int value);
}
```

Here is the improved version of the `repeat` method:

```
public static void repeat(int n, IntConsumer action) {
    for (int i = 0; i < n; i++) action.accept(i);
}
```

And here is how you call it:

```
repeat(10, i -> IO.println("Countdown: " + (9 - i)));
```

Table 6.2: Common Functional Interfaces

Functional Interface	Parameter Types	Return Type	Abstract Method Name	Description	Other Methods
`Runnable`	none	`void`	`run`	Runs an action without parameters or return value	
`Supplier<T>`	none	`T`	`get`	Supplies a value of type `T`	
`Consumer<T>`	`T`	`void`	`accept`	Consumes a value of type `T`	`andThen`

Functional Interface	Parameter Types	Return Type	Abstract Method Name	Description	Other Methods
`BiConsumer<T, U>`	`T`, `U`	`void`	`accept`	Consumes values of types `T` and `U`	`andThen`
`Function<T, R>`	`T`	`R`	`apply`	A function with parameter of type `T`	`compose`, `andThen`, `identity`
`BiFunction<T, U, R>`	`T`, `U`	`R`	`apply`	A function with parameters of types `T` and `U`	`andThen`
`UnaryOperator<T>`	`T`	`T`	`apply`	A unary operator on the type `T`	`compose`, `andThen`, `identity`
`BinaryOperator<T>`	`T`, `T`	`T`	`apply`	A binary operator on the type `T`	`andThen`, `maxBy`, `minBy`
`Predicate<T>`	`T`	`boolean`	`test`	A `boolean`-valued function	`and`, `or`, `negate`, `isEqual`, `not`
`BiPredicate<T, U>`	`T`, `U`	`boolean`	`test`	A `boolean`-valued function with two parameters	`and`, `or`, `negate`

Table 6.3 lists the 34 available specializations for primitive types `int`, `long`, and `double`. As you will see in Chapter 8, it is more efficient to use these specializations than the generic interfaces. For that reason, I used an `IntConsumer` instead of a `Consumer<Integer>` in the example of the preceding section.

Table 6.3: Functional Interfaces for Primitive Tvpes
p, q is `int`, `long`, `double`; P, Q is `Int`, `Long`, `Double`

Functional Interface	Parameter Types	Return Type	Abstract Method Name
`BooleanSupplier`	none	`boolean`	`getAsBoolean`
`PSupplier`	none	`p`	`getAsP`
`PConsumer`	`p`	`void`	`accept`
`ObjPConsumer<T>`	`T`, `p`	`void`	`accept`
`PFunction<T>`	`p`	`T`	`apply`
`PToQFunction`	`p`	`q`	`applyAsQ`
`ToPFunction<T>`	`T`	`p`	`applyAsP`
`ToPBiFunction<T, U>`	`T`, `U`	`p`	`applyAsP`
`PUnaryOperator`	`p`	`p`	`applyAsP`
`PBinaryOperator`	`p`, `p`	`p`	`applyAsP`
`PPredicate`	`p`	`boolean`	`test`



Tip: Use the standard interfaces for function types whenever you can. For example, suppose you write a method to process files that match a certain criterion. There is a legacy interface `java.io.FileFilter`. But if you use the standard `Predicate<File>` interface, you can take advantage of methods for creating, adapting, and combining predicates. The only reason not to do so would be if you already have many useful methods producing `FileFilter` instances.



Note: Most of the standard functional interfaces have nonabstract methods for producing or combining functions. For example, `Predicate.isEqual(a)` is the same as `a::equals`, but it also works if `a` is `null`. There are default methods `and`, `or`, `negate` for combining predicates. For example, `Predicate.isEqual(a).or(Predicate.isEqual(b))` is the same as `x -> a.equals(x) || b.equals(x)`.



Note: If you design your own interface with a single abstract method, you can tag it with the `@FunctionalInterface` annotation. This has two advantages. The compiler gives an error message if you accidentally add another abstract method. And the javadoc page includes a statement that your interface is a functional interface.

It is not required to use the annotation. Any interface with a single abstract method is, by definition, a functional interface. But using the `@FunctionalInterface` annotation is a good idea.



Note: Some programmers love chains of method calls, such as

```
String input = " 618970019642690137449562111 ";
boolean isPrime = input.strip().transform(BigInteger::new).isProbablePrime(20);
```

The `transform` method of the `String` class (added in Java 12) applies a `Function` to the string and yields the result. You could have equally well written

```
boolean prime = new BigInteger(input.strip()).isProbablePrime(20);
```

But then your eyes jump inside-out and left-to-right to find out what happens first and what happens next: Calling `strip`, then constructing the `BigInteger`, and finally testing if it is a probable prime.

I am not sure that the eyes-jumping-inside-out-and-left-to-right is a huge problem. But if you prefer the orderly left-to-right sequence of chained method calls, then `transform` is your friend.

Sadly, it only works for strings. Why isn't there a `transform(java.util.function.Function)` method in the `Object` class?

The Java API designers weren't fast enough. They had one chance to do this right—in Java 8, when the `java.util.function.Function` interface was added to the API. Up to that point, nobody could have added a `transform(java.util.function.Function)` method to their own classes. But in Java 12, it was too late. Someone somewhere could have defined `transform(java.util.function.Function)` in their class, with a different meaning. Admittedly, it is unlikely that this ever happened, but there is no way to know.

That is how Java works. It takes its commitments seriously, and won't renege on them for convenience.

6.2.10. Creating Comparators

The `Comparator` interface has a number of convenient static methods for creating comparators. These methods are intended to be used with lambda expressions or method references.

The static `comparing` method takes a "key extractor" function that maps a type `T` to a comparable type (such as `String`). The function is applied to the objects

to be compared, and the comparison is then made on the returned keys. For example, suppose you have an array of `Person` objects. Here is how you can sort them by name:

```
Arrays.sort(people, Comparator.comparing(Person::getName));
```

This is certainly much easier than implementing a `Comparator` by hand. Moreover, the code is clearer since it is obvious that we want to compare people by name.

You can chain comparators with the `thenComparing` method for breaking ties. For example,

```
Arrays.sort(people,
    Comparator.comparing(Person::getLastName)
        .thenComparing(Person::getFirstName));
```

If two people have the same last name, then the second comparator is used.

There are a few variations of these methods. You can specify a comparator to be used for the keys that the `comparing` and `thenComparing` methods extract. For example, here we sort people by the length of their names:

```
Arrays.sort(people, Comparator.comparing(Person::getName,
    (s, t) -> Integer.compare(s.length(), t.length())));
```

Moreover, both the `comparing` and `thenComparing` methods have variants that avoid boxing of `int`, `long`, or `double` values:

```
Arrays.sort(people, Comparator.comparing(Person::getName,
    Comparator.comparingInt(String::length)))
```

A shorter but perhaps less elegant way of producing the preceding operation would be:

```
Arrays.sort(people, Comparator.comparingInt(p -> p.getName().length()));
```

If your key function can return `null`, you will like the `nullsFirst` and `nullsLast` adapters. These static methods take an existing comparator and modify it so that it doesn't throw an exception when encountering `null` values but ranks them as smaller or larger than regular values. For example, suppose `getMiddleName` returns a `null` when a person has no middle name. Then you can use `Comparator.comparing(Person::getMiddleName, Comparator.nullsFirst(. . .))`.

The `nullsFirst` method needs a comparator—in this case, one that compares two strings. The `naturalOrder` method makes a comparator for any class implementing `Comparable`. A `Comparator.<String>naturalOrder()` is what we need. (See **Chapter 8** for an explanation of this syntax. Fortunately, the generic type can usually be inferred.) Here is the complete call for sorting by potentially null middle names. I use a static import of `java.util.Comparator.*`, to make the expression more legible.

```
Arrays.sort(people, comparing(Person::getMiddleName, nullsFirst(naturalOrder())));
```

The static `reverseOrder` method gives the reverse of the natural order. To reverse any comparator, use the `reversed` instance method. For example, `naturalOrder().reversed()` is the same as `reverseOrder()`.

6.3. Inner Classes

An *inner class* is a class that is defined inside another class. Why would you want to do that? There are two reasons:

- Inner classes can be hidden from other classes in the same package.
- Inner class methods can access the data from the scope in which they are defined—including the data that would otherwise be private.

Inner classes used to be very important for concisely implementing callbacks, but nowadays lambda expressions do a much better job. Still, inner classes can be very useful for structuring your code. The following sections walk you through all the details.



Note: C++ has *nested classes*. A nested class is contained inside the scope of the enclosing class. Here is a typical example: A linked list class defines a nested class to hold the nodes.

```
template<typename T>
class LinkedList {
public:
    class Node { // a nested class
    public:
        . . .
    private:
        T data;
        Node* next;
     };
     . . .
private:
    Node* head;
    Node* tail;
};
```

Nested classes are similar to inner classes in Java. However, the Java inner classes have an additional feature that makes them richer and more useful than nested classes in C++. An object that comes from an inner class has an implicit reference to the outer class object that instantiated it. Through this pointer, it gains access to the total state of the outer object. For example, in Java, the `Iterator` class would not need an explicit pointer to the `LinkedList` into which it points.

In Java, nested classes that are declared as `static` do not have this added pointer. They are the Java analog to nested classes in C++.

6.3.1. Use of an Inner Class to Access Object State

The syntax for inner classes is rather complex. For that reason, I present a simple but somewhat artificial example to demonstrate the use of inner classes. Let's refactor the `TimerDemo` example and extract a `TalkingClock` class. The constructor for a talking clock has two parameters: the interval between announcements and a flag to turn beeps on or off.

```
public class TalkingClock {
    private int interval;
    private boolean beep;

    public TalkingClock(int interval, boolean beep) { . . . }
    public void start() { . . . }

    public class TimePrinter implements ActionListener {
        // an inner class
        . . .
    }
}
```

Note that the `TimePrinter` class is now located inside the `TalkingClock` class. This does *not* mean that every `TalkingClock` has a `TimePrinter` instance field. As you will see, the `TimePrinter` objects are constructed by methods of the `TalkingClock` class.

Here is the `TimePrinter` class in greater detail. Note that the `actionPerformed` method checks the `beep` flag before emitting a beep.

```
public class TimePrinter implements ActionListener {
    public void actionPerformed(ActionEvent event) {
        IO.println("At the tone, the time is "
            + Instant.ofEpochMilli(event.getWhen()));
        if (beep) Toolkit.getDefaultToolkit().beep();
    }
}
```

Something surprising is going on. The `TimePrinter` class has no instance field or variable named `beep`. Instead, `beep` refers to the field of the `TalkingClock` object that created this `TimePrinter`. As you can see, an inner class method gets to access both its own instance fields *and* those of the outer object creating it.

For this to work, an object of an inner class always gets an implicit reference to the object that created it (see **Figure 6.3**).

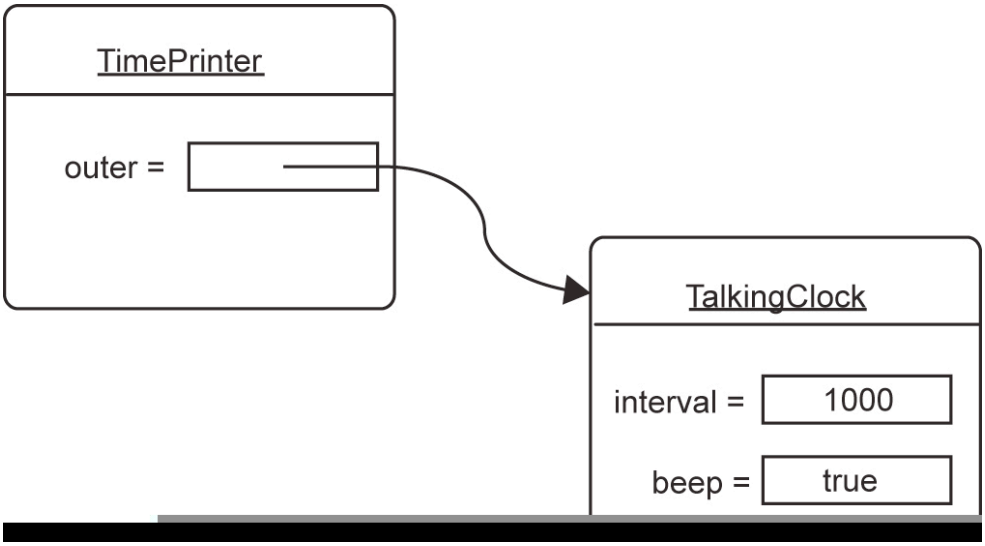


Figure 6.3: An inner class object has a reference to an outer class object.

This reference is invisible in the definition of the inner class. However, to illuminate the concept, let us call the reference to the outer object *outer*. Then the `actionPerformed` method is equivalent to the following:

```
public void actionPerformed(ActionEvent event) {
    IO.println("At the tone, the time is "
        + Instant.ofEpochMilli(event.getWhen()));
     if (outer.beep) Toolkit.getDefaultToolkit().beep();
}
```

The outer class reference is set in the constructor. The compiler modifies all inner class constructors, adding a parameter for the outer class reference. The `TimePrinter` class defines no constructors; therefore, the compiler synthesizes a no-argument constructor, generating code like this:

```
public TimePrinter(TalkingClock clock) { // automatically generated code
    outer = clock;
}
```

Again, please note that `outer` is not a Java keyword. We just use it to illustrate the mechanism involved in an inner class.

When a `TimePrinter` object is constructed in the `start` method, the compiler passes the `this` reference to the current talking clock into the constructor:

```
var listener = new TimePrinter(this); // parameter automatically added
```

Listing 6.7 shows the complete program that tests the inner class. Have another look at the access control. Had the `TimePrinter` class been a regular class, it would have needed to access the `beep` flag through a public method of the `TalkingClock` class. Using an inner class is an improvement. There is no need to provide accessors that are of interest only to one other class.



Note: We could have declared the `TimePrinter` class as `private`. Then only `TalkingClock` methods would be able to construct `TimePrinter` objects. Only inner classes can be private. Regular classes always have either package or public access.

Listing 6.7 v1ch06/innerClass/InnerClassDemo.java

```
1  package v1ch06.innerClass;
2
3  import module java.desktop;
4
5  /**
6   * This program demonstrates the use of inner classes.
7   */
8  class InnerClassDemo {
9      void main() {
10         var clock = new TalkingClock(1000, true);
11         clock.start();
12
13         // keep program running until the user selects "OK"
14         JOptionPane.showMessageDialog(null, "Quit program?");
15         System.exit(0);
16     }
17 }
```

Listing 6.8 v1ch06/innerClass/TalkingClock.java

```
1  package v1ch06.innerClass;
2
3  import module java.base;
4  import module java.desktop;
5  import javax.swing.Timer;
6
7  /**
8   * A clock that prints the time in regular intervals.
9   */
10 public class TalkingClock {
11     private int interval;
12     private boolean beep;
13
14     /**
15      * Constructs a talking clock.
16      * @param interval the interval between messages (in milliseconds)
```

```
     * @param beep true if the clock should beep
     */
    public TalkingClock(int interval, boolean beep) {
        this.interval = interval;
        this.beep = beep;
    }

    /**
     * Starts the clock.
     */
    public void start() {
        var listener = new TimePrinter();
        var timer = new Timer(interval, listener);
        timer.start();
    }

    public class TimePrinter implements ActionListener {
        public void actionPerformed(ActionEvent event) {
            IO.println("At the tone, the time is " + Instant.ofEpochMilli(event.get
            if (beep) Toolkit.getDefaultToolkit().beep();
        }
    }
}
```

6.3.2. Special Syntax Rules for Inner Classes

In the preceding section, we explained the outer class reference of an inner class by calling it *outer*. Actually, the proper syntax for the outer reference is a bit more complex. The expression

```
OuterClass.this
```

denotes the outer class reference. For example, you can write the `actionPerformed` method of the `TimePrinter` inner class as

```
public void actionPerformed(ActionEvent event) {
    . . .
    if (TalkingClock.this.beep) Toolkit.getDefaultToolkit().beep();
}
```

Conversely, you can write the inner object constructor more explicitly, using the syntax

```
outerObject.new InnerClass(construction arguments)
```

For example:

```
ActionListener listener = this.new TimePrinter();
```

Here, the outer class reference of the newly constructed `TimePrinter` object is set to the `this` reference of the method that creates the inner class object. This is the most common case. As always, the `this.` qualifier is redundant. However, it is also possible to set the outer class reference to another object by explicitly naming it. For example, since `TimePrinter` is a public inner class, you can construct a `TimePrinter` for any talking clock:

```
var jabberer = new TalkingClock(1000, true);
TalkingClock.TimePrinter listener = jabberer.new TimePrinter();
```

Note that you refer to an inner class as

```
OuterClass . InnerClass
```

when it occurs outside the scope of the outer class.



Note: As of Java 16, inner classes can have `static` members. Previously, `static` methods in inner classes were disallowed, and static fields declared in an inner class had to be final and initialized with a compile-time constant.

Static methods of an inner class can access static fields and methods from the inner class or enclosing classes.

6.3.3. Are Inner Classes Useful? Actually Necessary? Secure?

When inner classes were added to the Java language in Java 1.1, many programmers considered them a major new feature that was out of character with the Java philosophy of being simpler than C++. The inner class syntax is undeniably complex. (It gets more complex as we study anonymous inner classes later in this chapter.) It is not obvious how inner classes interact with other features of the language, such as access control and security.

Inner classes are translated into regular class files with `$` (dollar signs) separating the outer and inner class names. For example, the `TimePrinter` class inside the `innerClass.TalkingClock` class is translated to a class file `innerClass.TalkingClock$TimePrinter.class`. To see this at work, try the following experiment: run the `ReflectionDemo` program of **Chapter 5**, and give it the class `innerClass.TalkingClock$TimePrinter` to reflect upon. Alternatively, use the `javap` utility:

```
javap -private ClassName
```



Note: If you use UNIX, remember to escape the `$` character when you supply the class name on the command line. That is, run the `ReflectionDemo` or `javap` program as

```
java v1ch05/ReflectionDemo.java v1ch06.innerClass.TalkingClock\$TimePrinter
```

or

```
javap -private v1ch06.innerClass.TalkingClock\$TimePrinter
```

You will get the following printout:

```
public class v1ch06.innerClass.TalkingClock$TimePrinter
        implements java.awt.event.ActionListener {
    final v1ch06.innerClass.TalkingClock this$0;
    public v1ch06.innerClass.TalkingClock$TimePrinter(v1ch06.innerClass.TalkingClock);
    public void actionPerformed(java.awt.event.ActionEvent);
}
```

You can plainly see that the compiler has generated an additional instance field, `this$0`, for the reference to the outer class. (The name `this$0` is synthesized by the compiler—you cannot refer to it in your code.) You can also see the `TalkingClock` parameter for the constructor.



Note: Since Java 18, the `this$0` field is only provided when it is actually needed. It is dropped if no methods of the inner class access the outer class.

If the compiler can automatically do this transformation, couldn't you simply program the same mechanism by hand? Let's try it. We would make `TimePrinter` a regular class, outside the `TalkingClock` class. When constructing a `TimePrinter` object, we pass it the `this` reference of the object that is creating it.

```
class TalkingClock {
    . . .
    public void start() {
       var listener = new TimePrinter(this);
       var timer = new Timer(interval, listener);
       timer.start();
    }
}

class TimePrinter implements ActionListener {
    private TalkingClock outer;
    . . .
    public TimePrinter(TalkingClock clock) {
        outer = clock;
    }
}
```

Now let us look at the `actionPerformed` method. It needs to access `outer.beep`.

```
if (outer.beep) . . . // ERROR
```

Here we run into a problem. The inner class can access the private data of the outer class, but our external `TimePrinter` class cannot.

Thus, inner classes are genuinely more powerful than regular classes because they have more access privileges.

In Java 11, the virtual machine specification was updated to allow this “nest mate” access directly (see **https://openjdk.org/jeps/181**). In prior versions, the compiler generated helper methods.

6.3.4. Local Inner Classes

If you look carefully at the code of the `TalkingClock` example, you will find that you need the name of the type `TimePrinter` only once: when you create an object of that type in the `start` method.

In a situation like this, you can define the class *locally in a single method*.

```
public void start() {
    class TimePrinter implements ActionListener {
        public void actionPerformed(ActionEvent event) {
            IO.println("At the tone, the time is "
                + Instant.ofEpochMilli(event.getWhen()));
            if (beep) Toolkit.getDefaultToolkit().beep();
        }
    }

    var listener = new TimePrinter();
    var timer = new Timer(interval, listener);
    timer.start();
}
```

Local classes are never declared with an access specifier (that is, `public` or `private`). Their scope is always restricted to the block in which they are declared.

Local classes have one great advantage: They are completely hidden from the outside world—not even other code in the `TalkingClock` class can access them. No method except `start` has any knowledge of the `TimePrinter` class.

6.3.5. Accessing Variables from Outer Methods

Local classes have another advantage over other inner classes. Not only can they access the fields of their outer classes; they can even access local variables! However, those local variables must be *effectively* `final`. That means, they may never change once they have been assigned.

Here is a typical example. Let's move the `interval` and `beep` parameters from the `TalkingClock` constructor to the `start` method.

```
public void start(int interval, boolean beep) {
    class TimePrinter implements ActionListener {
        public void actionPerformed(ActionEvent event) {
            IO.println("At the tone, the time is "
                + Instant.ofEpochMilli(event.getWhen()));
            if (beep) Toolkit.getDefaultToolkit().beep();
        }
    }

    var listener = new TimePrinter();
    var timer = new Timer(interval, listener);
    timer.start();
}
```

Note that the `TalkingClock` class no longer needs to store a `beep` instance field. It simply refers to the `beep` parameter variable of the `start` method.

Maybe this should not be so surprising. The line

```
if (beep) . . .
```

is, after all, ultimately inside the `start` method, so why shouldn't it have access to the value of the `beep` variable?

To see why there is a subtle issue here, let's consider the flow of control more closely.

1. The `start` method is called.
2. The object variable `listener` is initialized by a call to the constructor of the inner class `TimePrinter`.
3. The `listener` reference is passed to the `Timer` constructor, the timer is started, and the `start` method exits. At this point, the `beep` parameter variable of the `start` method no longer exists.
4. A second later, the `actionPerformed` method executes `if (beep) . . .`

For the code in the `actionPerformed` method to work, the `TimePrinter` class must have copied the `beep` field as a local variable of the `start` method, before the `beep` parameter value went away. That is indeed exactly what happens. In our example, the compiler synthesizes the name `localInnerClass.TalkingClock$1TimePrinter` for the local inner class. If you use the `ReflectionDemo` program again to spy on the `localInnerClass.TalkingClock$1TimePrinter` class, you will get the following output:

```
class v1ch06.localInnerClass.TalkingClock$1TimePrinter {
    v1ch06.localInnerClass.TalkingClock$1TimePrinter(
        v1ch06.localInnerClass.TalkingClock, boolean);
    public void actionPerformed(java.awt.event.ActionEvent);
    final boolean val$beep;
}
```

When an object is created, the current value of the `beep` variable is stored in the `val$beep` field. The inner class field persists after the local variable goes out of scope.



Note: You can follow the virtual machine instructions that are used to initialize and read the `val$beep` field by running

```
javap -private -c v1ch06.localInnerClass.TalkingClock\$1TimePrinter
```

The variable is set in the constructor, before the call to the `Object` constructor. It is read in the `actionPerformed` method.

6.3.6. Anonymous Inner Classes

When using local inner classes, you can often go a step further. If you want to make only a single object of this class, you don't even need to give the class a name. Such a class is called an *anonymous inner class*.

```
public void start(int interval, boolean beep) {
    var listener = new ActionListener() {
        public void actionPerformed(ActionEvent event) {
            IO.println("At the tone, the time is "
                + Instant.ofEpochMilli(event.getWhen()));
            if (beep) Toolkit.getDefaultToolkit().beep();
        }
    };
    var timer = new Timer(interval, listener);
    timer.start();
}
```

This syntax is very cryptic indeed. What it means is this: Create a new object of a class that implements the `ActionListener` interface, where the required method `actionPerformed` is the one defined inside the braces `{ }`.

In general, the syntax is

```
new SuperType(construction arguments) {
    inner class methods and data
}
```

Here, `SuperType` can be an interface, such as `ActionListener`; then, the inner class implements that interface. `SuperType` can also be a class; then, the inner class extends that class.

An anonymous inner class cannot have constructors because the name of a constructor must be the same as the name of a class, and the class has no name. Instead, the construction arguments are given to the *superclass* constructor. In particular, whenever an inner class implements an interface, it cannot have any construction arguments. Nevertheless, you must supply a set of parentheses as in

```
new InterfaceType() {
    methods and data
}
```

You have to look carefully to see the difference between the construction of a new object of a class and the construction of an object of an anonymous inner class extending that class.

```
var queen = new Person("Mary");
    // a Person object
var count = new Person("Dracula") { . . . };
    // an object of an inner class extending Person
```

If the closing parenthesis of the construction argument list is followed by an opening brace, then an anonymous inner class is being defined.



Note: Even though an anonymous class cannot have constructors, you can provide an object initialization block:

```
var count = new Person("Dracula") {
    {
        initialization
    }
    . . .
};
```

Listing 6.9 contains the complete source code for the talking clock program with an anonymous inner class. If you compare this program with **Listing 6.7**, you will see that in this case, the solution with the anonymous inner class is quite a bit shorter and, hopefully, with some practice, as easy to comprehend.

For many years, Java programmers routinely used anonymous inner classes for event listeners and other callbacks. Nowadays, you are better off using a lambda expression. For example, the `start` method from the beginning of this section can be written much more concisely with a lambda expression like this:

```
public void start(int interval, boolean beep) {
    var timer = new Timer(interval, event -> {
        IO.println( "At the tone, the time is "
            + Instant.ofEpochMilli(event.getWhen()));
        if (beep) Toolkit.getDefaultToolkit().beep();
    });
    timer.start();
}
```



Note: If you store an anonymous class instance in a variable defined with `var`, the variable knows about added methods or fields:

```
var bob = new Object() { String name = "Bob"; }
IO.println(bob.name);
```

The object constructed with `new Object() { String name = "Bob"; }` has type "`Object` with a `String name` field." This is a *nondenotable* type—a type that you cannot express with Java syntax. Nevertheless, the compiler understands the type, and it becomes the type for the `bob` variable.

If you declare `bob` as having type `Object`, then `bob.name` does not compile.



Note: The following trick, called *double brace initialization*, takes advantage of the inner class syntax. Suppose you want to construct an array list and pass it to a method:

```
var friends = new ArrayList<String>();
friends.add("Harry");
friends.add("Tony");
invite(friends);
```

If you don't need the array list again, it would be nice to make it anonymous. But then how can you add the elements? Here is how:

```
invite(new ArrayList<String>() {{ add("Harry"); add("Tony"); }});
```

Note the double braces. The outer braces make an anonymous subclass of `ArrayList`. The inner braces are an object initialization block (see **Chapter 4**).

In practice, this trick is rarely useful. More likely than not, the `invite` method is willing to accept any `List<String>`, and you can simply pass `List.of("Harry", "Tony")`.



Caution: It is often convenient to make an anonymous subclass that is almost, but not quite, like its superclass. But you need to be careful with the `equals` method. In **Chapter 5**, I recommended that your `equals` methods use a test

```
if (getClass() != other.getClass()) return false;
```

An anonymous subclass will fail this test.



Tip: When you produce logging or debugging messages, you often want to include the name of the current class, such as

```
System.err.println("Something awful happened in " + getClass());
```

But that fails in a static method. After all, the call to `getClass` calls `this.getClass()`, and a static method has no `this`. Use the following expression instead:

```
new Object(){}.getClass().getEnclosingClass() // gets class of static method
```

Here, `new Object(){}` makes an anonymous object of an anonymous subclass of `Object`, and `getEnclosingClass` gets its enclosing class—that is, the class containing the static method.

Listing 6.9 v1ch06/anonymousInnerClass/AnonymousInnerClassDemo.java

```
1  package v1ch06.anonymousInnerClass;
2
3  import module java.desktop;
4
5  /**
6   * This program demonstrates anonymous inner classes.
7   */
8  class AnonymousInnerClassDemo {
9      void main() {
10         var clock = new TalkingClock();
11         clock.start(1000, true);
12
13         // keep program running until the user selects "OK"
14         JOptionPane.showMessageDialog(null, "Quit program?");
15         System.exit(0);
16     }
17 }
```

Listing 6.10 v1ch06/anonymousInnerClass/TalkingClock.java

```
1  package v1ch06.anonymousInnerClass;
2
3  import module java.base;
4  import module java.desktop;
5  import javax.swing.Timer;
6
7  /**
8   * A clock that prints the time in regular intervals.
9   */
10 public class TalkingClock {
11     /**
12      * Starts the clock.
13      * @param interval the interval between messages (in milliseconds)
14      * @param beep true if the clock should beep
15      */
16     public void start(int interval, boolean beep) {
17         var listener = new ActionListener() {
18             public void actionPerformed(ActionEvent event) {
19                 IO.println(
20                         "At the tone, the time is " + Instant.ofEpochMilli(event.ge
21                 if (beep) Toolkit.getDefaultToolkit().beep();
22             }
23         };
24         var timer = new Timer(interval, listener);
25         timer.start();
26     }
27 }
```

6.3.7. Static Classes

Occasionally, you may want to nest one class inside another, but you don't need the nested class to have a reference to the outer class object. You can suppress the generation of that reference by declaring the nested class `static`.

The Java Language Specification uses the term "nested class" for any class that is declared inside another class or interface, "static class" for a (necessarily nested) `static` class, and "inner class" for a nested class that is not `static`.

Here is a typical example of where you would want to do this. In an `ArrayAlg` class, we have a task that finds a range of elements of an array. Then you need to return the start and the end of the range. We can achieve that by defining a class `Range` that holds two values:

```
class Range {
    private int from;
    private int to;

    public Range(int from) { . . . }
```

```
    public void extend() { . . . }
    . . .
}
```

Of course, `Range` is an exceedingly common name, and in a large project, it is quite possible that some other programmer had the same bright idea and defined another `Range` class in the same package. We can solve this potential name clash by making `Range` a public inner class inside `ArrayAlg`. Then the class will be known to the public as `ArrayAlg.Range`:

```
ArrayAlg.Range r = ArrayAlg.longestRun(numbers);
```

However, unlike the inner classes used in previous examples, we do not want to have a reference to any other object inside a `Range` object. That reference can be suppressed by declaring the nested class `static`:

```
class ArrayAlg {
    public static class Range {
        . . .
    }
    . . .
}
```

A static class is exactly like an inner class, except that an object of a static class does not have a reference to the outer class object that generated it.

In our example, we must use a static class because the nested class instance is constructed inside a static method:

```
public static Pair longestRun(double[] values) {
    . . .
    Range current = new Range(. . .);
    . . .
}
```

Had the `Range` class not been declared as `static`, the compiler would have flagged the constructor call as an error. After all, there is no object of type `ArrayAlg` available to initialize the inner class instance.

You should use a static class whenever a nested class does not need to access an outer class object.



Note: Classes that are declared inside an interface are automatically `static`. They could not possibly hold a reference to an outer class object, because its type would be unknowable. Any number of classes can implement the interface.



Note: Prior to Java 16, it was not possible to declare a `static` class inside an inner class. This restriction was never necessary and has now been removed.

Listing 6.11 contains the complete source code of the `ArrayAlg` class and the nested `Pair` class.

Listing 6.11 v1ch06/StaticInnerClassDemo.java

```
1  package v1ch06;
2
3  /**
4   * This program demonstrates the use of static inner classes.
5   */
6  class StaticInnerClassDemo {
7      void main() {
8          double[] numbers = { 1, 2, 2, 3, 3, 3, 4, 4, 4, 4, 5, 5, 5, 5, 5, 6, 6, 6,
9          ArrayAlg.Range r = ArrayAlg.longestRun(numbers);
10         IO.println("from = " + r.getFrom());
11         IO.println("to = " + r.getTo());
12     }
13
14 }
15
16 class ArrayAlg {
17     /**
18      * A range of index values.
19      */
20     public static class Range {
21         private int from;
22         private int to;
23
24         /**
25          * Constructs a range of length 1.
26          *
27          * @param from the initial index value of this range
28          */
29         public Range(int from) {
30             this.from = from;
31             this.to = from + 1;
32         }
33
34         /**
35          * Extends this range by one element.
36          */
37         public void extend() {
38             this.to++;
39         }
40
41         /**
42          * Gets the starting index value of this range.
43          * @return the starting index
44          */
45         public int getFrom() {
```

```
46              return from;
47          }
48
49          /**
50           * Gets the first index past the end of this range.
51           * @return the past-the-end index
52           */
53          public int getTo() {
54              return to;
55          }
56
57          /**
58           * Returns the number of elements in this range.
59           * @return the number of elements
60           */
61          public int length() {
62              return to - from;
63          }
64      }
65
66      /**
67       * A "run" is a sequence of repeating adjacent elements. For example, in the
68       * array 1 2 3 3 3 4 4, the runs are (trivially) 1 and 2, and 3 3 3 3 and 4
69       * 4. Returns the range of the longest run.
70       *
71       * @param values an array of length at least 1
72       * @return the range of the longest run
73       */
74      public static Range longestRun(double[] values) {
75          Range longest = new Range(0);
76          Range current = new Range(0);
77          for (int i = 1; i < values.length; i++) {
78              if (values[i] == values[i - 1])
79                  current.extend();
80              else {
81                  if (longest.length() < current.length()) longest = current;
82                  current = new Range(i);
83              }
84          }
85          if (longest.length() < current.length()) longest = current;
86          return longest;
87      }
88  }
```

6.3.8. Nested Records

In the preceding example, I purposefully made the `Range` class mutable. It might be better to make the `Range` class immutable, and to declare it as a record.

```
class ArrayAlg {
    record Range(int from, int to) {}
    . . .
}
```

A nested record is automatically `static`. For example, this `Range` record has exactly two components, the integers `from` and `to`, and no reference to an outer class object.

With compact compilation units, the static nature of records can be unintuitive. Consider a program like this:

```
int[] values;
void main() { . . . }
record Range(int from, int to) { // Nested in the implicitly declared class
    int sum() {
        int result = 0;
        for (int i = from; i <= to; i++) {
            result += values[i]; // ERROR
        }
        return result;
    }
}
```

It looks as if the `sum` method could access the "global" `values` variable. But that is an *instance* variable of the enclosing implicit class. And the methods of the `static` nested record have no access to enclosing instance variables.



Note: A nested enumeration is automatically `static`, just like a nested record. The instances of the enumeration do not have references to any outer class object. Similarly, nested interfaces are implicitly `static`, since interfaces have no data.

6.4. Service Loaders

Sometimes, you develop an application with a service architecture. There are platforms that encourage this approach, such as OSGi (**https://osgi.org**), which are used in development environments, application servers, and other complex applications. Such platforms go well beyond the scope of this book, but the JDK also offers a simple mechanism for loading services, described here. This mechanism is well supported by the Java Platform Module System—see **Chapter 12**.

Often, when providing a service, a program wants to give the service designer some freedom of how to implement the service's features. It can also be desirable to have multiple implementations to choose from. The `ServiceLoader` class makes it easy to load services that conform to a common interface.

Define an interface (or, if you prefer, a superclass) with the methods that each instance of the service should provide. For example, suppose your service provides encryption.

```
package serviceLoader;

public interface Cipher {
```

```
    byte[] encrypt(byte[] source, byte[] key);
    byte[] decrypt(byte[] source, byte[] key);
    int strength();
}
```

The service provider supplies one or more classes that implement this service, for example

```
package serviceLoader.impl;

public class CaesarCipher implements Cipher {
    public byte[] encrypt(byte[] source, byte[] key) {
        var result = new byte[source.length];
        for (int i = 0; i < source.length; i++)
            result[i] = (byte)(source[i] + key[0]);
        return result;
    }

    public byte[] decrypt(byte[] source, byte[] key) {
        return encrypt(source, new byte[] { (byte) -key[0] });
    }

    public int strength() { return 1; }
}
```

The implementing classes can be in any package, not necessarily the same package as the service interface. Each of them must have a no-argument constructor.

Now add the names of the classes to a UTF-8 encoded text file in the `META-INF/services` directory whose name matches the qualified interface name. In our example, the file `META-INF/services/serviceLoader.Cipher` would contain the line

```
serviceLoader.impl.CaesarCipher
```

In this example, we provide a single implementing class. You could also provide multiple classes and later pick among them.

With this preparation done, the program initializes a service loader as follows:

```
public static ServiceLoader<Cipher> cipherLoader = ServiceLoader.load(Cipher.class);
```

This should be done just once in the program.

The `iterator` method of the service loader returns an iterator through all provided implementations of the service. (See **Chapter 9** for more information about iterators.) It is easiest to use an enhanced `for` loop to traverse them. In the loop, pick an appropriate object to carry out the service.

```
public static Cipher getCipher(int minStrength) {
    for (Cipher cipher : cipherLoader) { // implicitly calls cipherLoader.iterator()
        if (cipher.strength() >= minStrength) return cipher;
    }
    return null;
}
```

Alternatively, you can use streams (see **Chapter 1 of Volume II**) to locate the desired service. The `stream` method yields a stream of `ServiceLoader.Provider` instances. That interface has methods `type` and `get` for getting the provider class and the provider instance. If you select a provider by type, then you just call `type` and no service instances are unnecessarily instantiated.

```
public static Optional<Cipher> getCipher2(int minStrength) {
    return cipherLoader.stream()
        .filter(descr -> descr.type() == serviceLoader.impl.CaesarCipher.class)
        .findFirst()
        .map(ServiceLoader.Provider::get);
}
```

Finally, if you are willing to take any service instance, simply call `findFirst`:

```
Optional<Cipher> cipher = cipherLoader.findFirst();
```

The `Optional` class is explained in **Chapter 1 of Volume II**.

`java.util.ServiceLoader<S>` 1.6

- `static <S> ServiceLoader<S> load(Class<S> service)`
  creates a service loader for loading the classes that implement the given service interface.
- `Iterator<S> iterator()`
  yields an iterator that lazily loads the service classes. That is, a class is loaded whenever the iterator advances.
- `Stream<ServiceLoader.Provider<S>> stream()` 9
  returns a stream of provider descriptors, so that a provider of a desired class can be loaded lazily.
- `Optional<S> findFirst()` 9
  finds the first available service provider, if any.

java.util.ServiceLoader.Provider<S> 9

- `Class<? extends S> type()`
  gets the type of this provider.

- `S get()`
  gets an instance of this provider.

6.5. Proxies

In the final section of this chapter, we discuss *proxies*. You can use a proxy to create, at runtime, new classes that implement a given set of interfaces. Proxies are only necessary when you don't yet know at compile time which interfaces you need to implement. This is not a common situation for application programmers, so feel free to skip this section if you are not interested in advanced wizardry. However, for certain systems programming applications, the flexibility that proxies offer can be very important.

6.5.1. When to Use Proxies

Suppose you want to construct an object of a class that implements one or more interfaces whose exact nature you may not know at compile time. This is a difficult problem. To construct an actual class, you can simply use the `newInstance` method or use reflection to find a constructor. But you can't instantiate an interface. You need to define a new class in a running program.

To overcome this problem, some programs generate code, place it into a file, invoke the compiler, and then load the resulting class file. Naturally, this is slow, and it also requires deployment of the compiler together with the program. The *proxy* mechanism is a better solution. The proxy class can create brand-new classes at runtime. Such a proxy class implements the interfaces that you specify. In particular, the proxy class has the following methods:

- All methods required by the specified interfaces; and
- All methods defined in the `Object` class (`toString`, `equals`, and so on).

However, you cannot define new code for these methods at runtime. Instead, you must supply an *invocation handler*. An invocation handler is an object of any class that implements the `InvocationHandler` interface. That interface has a single method:

```
Object invoke(Object proxy, Method method, Object[] args)
```

Whenever a method is called on the proxy object, the `invoke` method of the invocation handler gets called, with the `Method` object and arguments of the original call. The invocation handler must then figure out how to handle the call.

6.5.2. Creating Proxy Objects

To create a proxy object, use the `newProxyInstance` method of the `Proxy` class. The method has three parameters:

- A *class loader*. As part of the Java security model, different class loaders can be used for platform and application classes, classes that are downloaded from the Internet, and so on. We will discuss class loaders in **Chapter 9 of Volume II**. In this example, we specify the "system class loader" that loads platform and application classes.
- An array of `Class` objects, one for each interface to be implemented.
- An invocation handler.

There are two remaining questions. How do we define the handler? And what can we do with the resulting proxy object? The answers depend, of course, on the problem that we want to solve with the proxy mechanism. Proxies can be used for many purposes, such as

- Routing method calls to remote servers
- Associating user interface events with actions in a running program
- Tracing method calls for debugging purposes

In our example program, we use proxies and invocation handlers to trace method calls. We define a `TraceHandler` wrapper class that stores a wrapped object. Its `invoke` method simply prints the name and arguments of the method to be called and then calls the method with the wrapped object as the implicit argument.

```
class TraceHandler implements InvocationHandler {
    private Object target;

    public TraceHandler(Object t) {
        target = t;
    }

    public Object invoke(Object proxy, Method m, Object[] args)
            throws Throwable {
        // print method name and arguments
        . . .
        // invoke actual method
        return m.invoke(target, args);
    }
}
```

Here is how you construct a proxy object that causes the tracing behavior whenever one of its methods is called:

```
Object value = . . .;
// construct wrapper
var handler = new TraceHandler(value);
// construct proxy for one or more interfaces
var interfaces = new Class[] { Comparable.class };
Object proxy = Proxy.newProxyInstance(
    ClassLoader.getSystemClassLoader(),
    new Class[] { Comparable.class }, handler);
```

Now, whenever a method from one of the interfaces is called on `proxy`, the method name and arguments are printed out and the method is then invoked on `value`.

In the program shown in **Listing 6.12**, we use proxy objects to trace a binary search. We fill an array with proxies to the integers `1 . . . 1000`. Then we invoke the `binarySearch` method of the `Arrays` class to search for a random integer in the array. Finally, we print the matching element.

```
var elements = new Object[1000];
// fill elements with proxies for the integers 1 . . . 1000
for (int i = 0; i < elements.length; i++) {
    Integer value = i + 1;
    elements[i] = Proxy.newProxyInstance(. . .); // proxy for value;
}

// construct a random integer
Integer key = (int) (Math.random() * elements.length) + 1;

// search for the key
int result = Arrays.binarySearch(elements, key);

// print match if found
if (result >= 0) IO.println(elements[result]);
```

The `Integer` class implements the `Comparable` interface. The proxy objects belong to a class that is defined at runtime. (It has a name such as `$Proxy0`.) That class also implements the `Comparable` interface. However, its `compareTo` method calls the `invoke` method of the proxy object's handler.



Note: As you saw earlier in this chapter, the `Integer` class actually implements `Comparable<Integer>`. However, at runtime, all generic types are erased and the proxy is constructed with the class object for the raw `Comparable` class.

The `binarySearch` method makes calls like this:

```
if (elements[i].compareTo(key) < 0) . . .
```

Since we filled the array with proxy objects, the `compareTo` calls the `invoke` method of the `TraceHandler` class. That method prints the method name and arguments and then invokes `compareTo` on the wrapped `Integer` object.

Finally, at the end of the sample program, we call

```
IO.println(elements[result]);
```

The `println` method calls `toString` on the proxy object, and that call is also redirected to the invocation handler.

Here is the complete trace of a program run:

```
500.compareTo(288)
250.compareTo(288)
375.compareTo(288)
312.compareTo(288)
281.compareTo(288)
296.compareTo(288)
288.compareTo(288)
288.toString()
```

You can see how the binary search algorithm homes in on the key by cutting the search interval in half in every step. Note that the `toString` method is proxied even though it does not belong to the `Comparable` interface—as you will see in the next section, certain `Object` methods are always proxied.

Listing 6.12 v1ch06/ProxyDemo.java

```
1  package v1ch06;
2
3  import module java.base;
4  import java.lang.reflect.Proxy;
5
6  /**
7   * This program demonstrates the use of proxies.
8   */
9  class ProxyDemo {
10     void main() {
11         var elements = new Object[1000];
12
13         // fill elements with proxies for the integers 1 . . . 1000
14         for (int i = 0; i < elements.length; i++) {
15             Integer value = i + 1;
16             var handler = new TraceHandler(value);
17             Object proxy = Proxy.newProxyInstance(ClassLoader.getSystemClassLoader(
18                     new Class[]{Comparable.class}, handler);
19             elements[i] = proxy;
20         }
21
22         // construct a random integer
23         Integer key = (int) (Math.random() * elements.length) + 1;
24
25         // search for the key
26         int result = Arrays.binarySearch(elements, key);
27
28         // print match if found
29         if (result >= 0) IO.println(elements[result]);
30     }
31 }
32
```

```
/**
 * An invocation handler that prints out the method name and parameters, then
 * invokes the original method.
 */
class TraceHandler implements InvocationHandler {
    private Object target;

    /**
     * Constructs a TraceHandler.
     *
     * @param t the implicit parameter of the method call
     */
    public TraceHandler(Object t) {
        target = t;
    }

    public Object invoke(Object proxy, Method m, Object[] args) throws Throwable {
        // print implicit argument
        IO.print(target);
        // print method name
        IO.print("." + m.getName() + "(");
        // print explicit arguments
        if (args != null) {
            for (int i = 0; i < args.length; i++) {
                IO.print(args[i]);
                if (i < args.length - 1) IO.print(", ");
            }
        }
        IO.println(")");

        // invoke actual method
        return m.invoke(target, args);
    }
}
```

6.5.3. Properties of Proxy Classes

Now that you have seen proxy classes in action, let's go over some of their properties. Remember that proxy classes are created on the fly in a running program. However, once they are created, they are regular classes, just like any other classes in the virtual machine.

All proxy classes extend the class `Proxy`. A proxy class has only one instance field—the invocation handler, which is defined in the `Proxy` superclass. Any additional data required to carry out the proxy objects' tasks must be stored in the invocation handler. For example, when we proxied `Comparable` objects in the program shown in **Listing 6.12**, the `TraceHandler` wrapped the actual objects.

All proxy classes override the `toString`, `equals`, and `hashCode` methods of the `Object` class. Like all proxy methods, these methods simply call `invoke` on the invocation handler. The other methods of the `Object` class (such as `clone` and `getClass`) are not redefined.

The names of proxy classes are not defined. The `Proxy` class in Oracle's virtual machine generates class names that begin with the string `$Proxy`.

There is only one proxy class for a particular class loader and ordered set of interfaces. That is, if you call the `newProxyInstance` method twice with the same class loader and interface array, you get two objects of the same class. You can also obtain that class with the `getProxyClass` method:

```
Class proxyClass = Proxy.getProxyClass(null, interfaces);
```

A proxy class is always `public` and `final`. If all interfaces that the proxy class implements are `public`, the proxy class does not belong to any particular package. Otherwise, all non-`public` interfaces must belong to the same package, and the proxy class will also belong to that package.

You can test whether a particular `Class` object represents a proxy class by calling the `isProxyClass` method of the `Proxy` class.



Note: Calling a default method of a proxy triggers the invocation handler. To actually invoke the method, use the static `invokeDefault` method of the `InvocationHandler` interface. For example, here is an invocation handler that calls the default methods and passes the abstract methods to another target:

```
InvocationHandler handler = (proxy, method, args) -> {
    if (method.isDefault())
        return InvocationHandler.invokeDefault(proxy, method, args)
    else
        return method.invoke(target, args);
};
```

---

java.lang.reflect.InvocationHandler **1.3**

- `Object invoke(Object proxy, Method method, Object[] args)`
  define this method to contain the action that you want carried out whenever a method was invoked on the proxy object.
- `static Object invokeDefault(Object proxy, Method method, Object... args)` **16**
  invokes a default method of the proxy instance with the given arguments, bypassing the invocation handler.

`java.lang.reflect.Proxy` **1.3**

- `static Class<?> getProxyClass(ClassLoader loader, Class<?>... interfaces)`
  returns the proxy class that implements the given interfaces.

- `static Object newProxyInstance(ClassLoader loader, Class<?>[] interfaces, InvocationHandler handler)`
  constructs a new instance of the proxy class that implements the given interfaces. All methods call the `invoke` method of the given handler object.
- `static boolean isProxyClass(Class<?> cl)`
  returns `true` if `cl` is a proxy class.

This ends the final chapter on the object-oriented features of the Java programming language. Interfaces, lambda expressions, and inner classes are concepts that you will encounter frequently, whereas cloning, service loaders, and proxies are advanced techniques that are of interest mainly to library designers and tool builders, not application programmers. You are now ready to learn how to deal with exceptional situations in your programs in **Chapter 7**.

Previous chapter
< 5. Inheritance

Next chapter
7. Exceptions, Assertions, and Logging >